N50 Dockerfile로 실행환경 정하기_해설서

Chapter01

챕터 01. Dockerfile로 실행 환경 정하기

1. 학습 목적

이번 강의의 핵심은 내 컴퓨터에서 잘 작동하는 AI 에이전트를 서버에서도 동일하게 실행할 수 있도록 실행 환경을 직접 정의하는 방법을 배우는 것입니다.

지금까지는 코드를 작성하고 테스트한 뒤 서버에 배포하는 과정을 중심으로 공부했다면, 이번에는 한 단계 더 나아가겠습니다. 코드뿐 아니라 그 코드가 실행되는 환경까지 관리하는 것입니다.

2. 중학생도 이해할 수 있는 설명

① 왜 내 컴퓨터에서는 되는데 서버에서는 안 될까요?

식당에서 요리한다고 생각해 보겠습니다.

- 내 컴퓨터: 요리사와 조리 도구, 재료가 모두 준비된 주방입니다.
- 서버: 똑같은 요리사가 있지만 필요한 조리 도구 하나가 없는 주방입니다.
- 코드: 요리사가 따라 하는 레시피입니다.

레시피가 똑같아도 필요한 도구가 없으면 요리를 완성할 수 없습니다.

AI 서비스도 마찬가지입니다. 코드는 동일하더라도 Python 버전, 설치된 라이브러리, 운영체제의 시스템 패키지 등이 다르면 실행 결과가 달라질 수 있습니다.

이번 강의의 사례에서는 사진 속 글자를 읽는 기능이 서버에서 실패합니다. 이미지 처리에 필요한 시스템 프로그램이 서버 실행 환경에 설치되지 않았을 가능성을 확인해야 합니다.

② buildpacks와 Dockerfile은 무엇이 다른가요?

buildpacks — 자동으로 준비하는 방식

소스 코드를 분석해 Python과 라이브러리 등 필요한 실행 환경을 자동으로 구성합니다. 편리하지만, 코드만으로 알기 어려운 운영체제 수준의 의존성까지 항상 원하는 대로 구성해 주

는 것은 아닙니다.

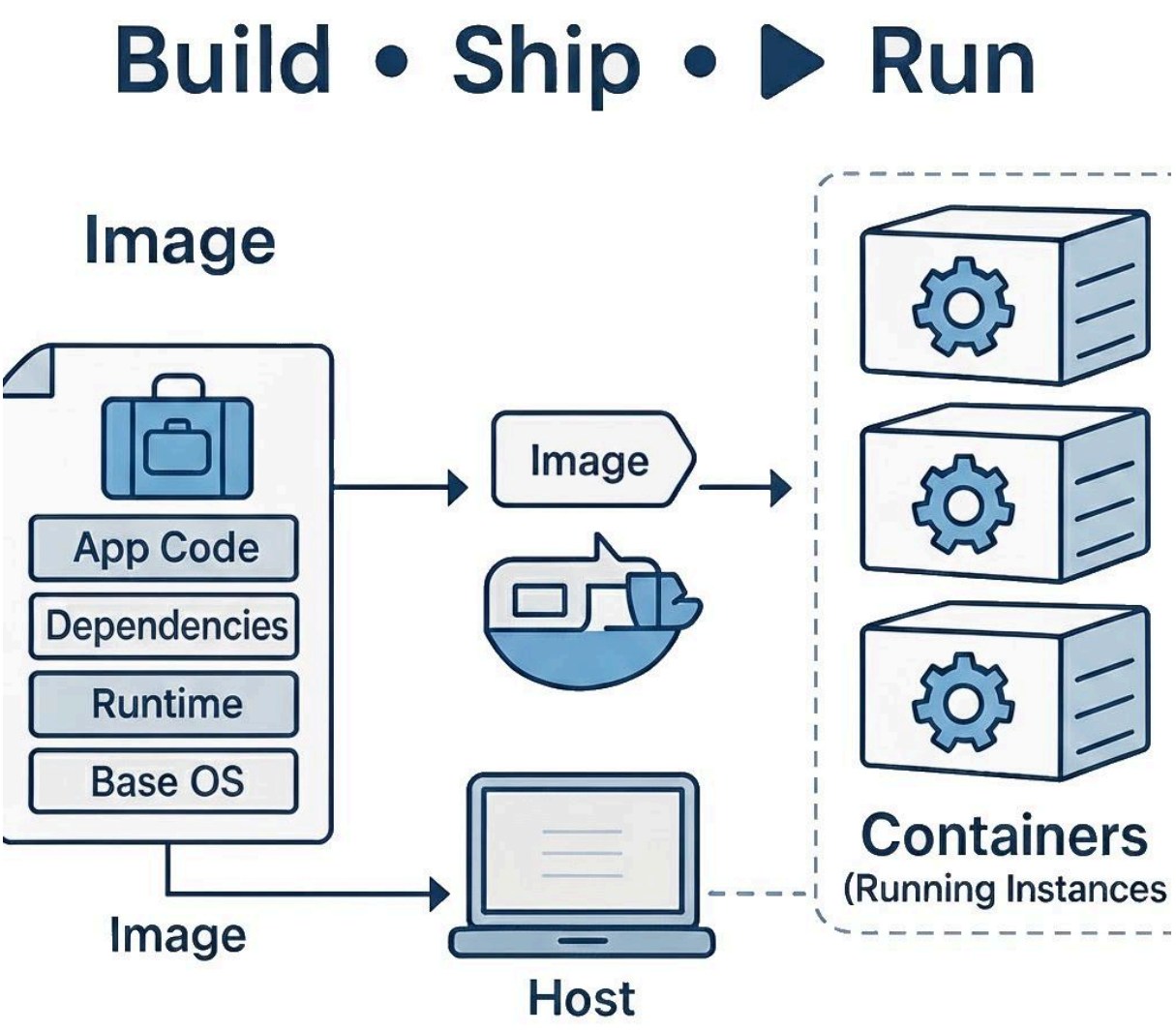
Dockerfile — 직접 지시하는 방식

어떤 기반 환경을 사용하고, 어떤 시스템 패키지를 설치하며, 어떤 명령으로 서비스를 실행할지 명시합니다.

비유하자면 buildpacks는 필요한 물건을 알아서 준비해 주는 서비스이고, Dockerfile은 준비할 물건과 순서를 직접 적어 놓은 상세 작업 지시서입니다.

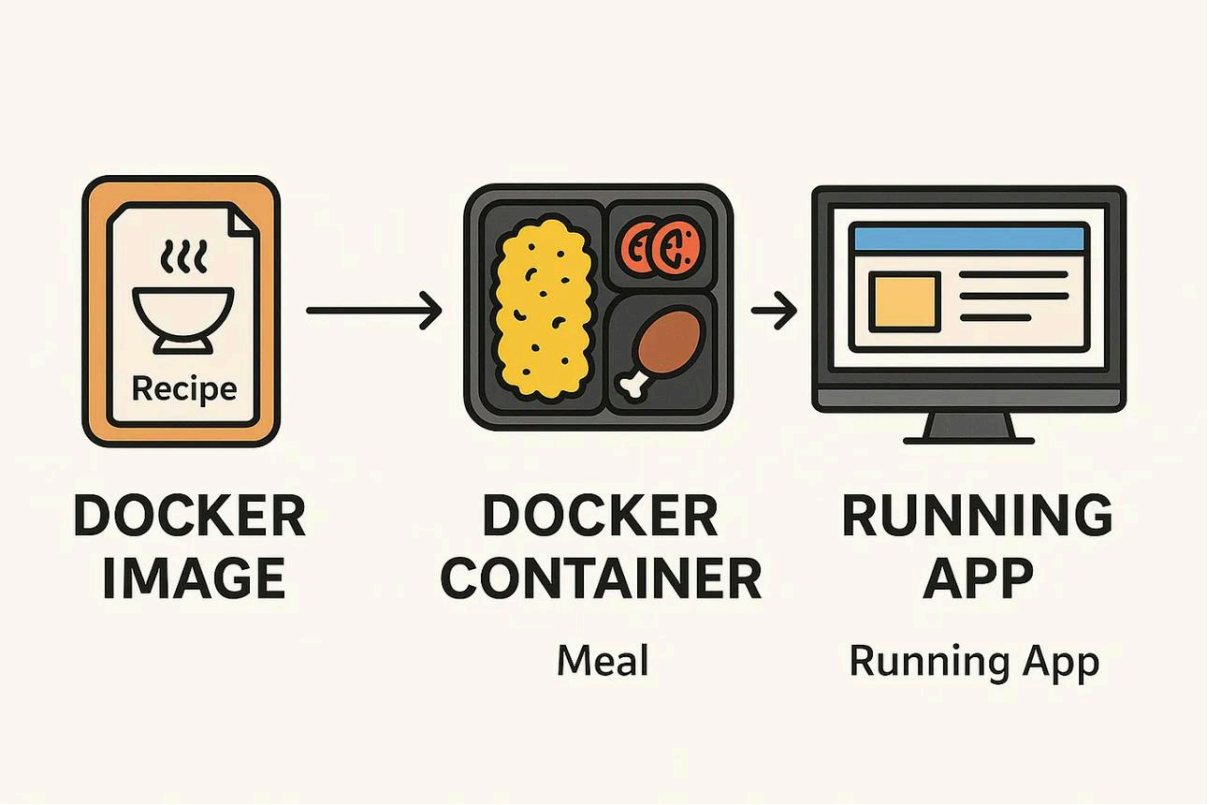
3. 이미지와 컨테이너의 차이

이 두 개념은 Docker를 이해하는 데 가장 중요합니다.



이미지(Image)

컨테이너를 실행하기 위한 환경과 파일을 묶어 놓은 실행용 패키지입니다. 애플리케이션 코드, 필요한 라이브러리, 시스템 패키지 등이 포함될 수 있습니다.



컨테이너(Container)

이미지를 바탕으로 실제 실행 중인 인스턴스입니다. 같은 이미지로 여러 컨테이너를 실행할 수도 있습니다.

기억하기 쉬운 비유는 다음과 같습니다.

- 이미지 = 붕어빵 틀과 재료를 갖춘 준비물
- 컨테이너 = 그 준비물을 사용해 실제로 만들어 낸 붕어빵

Dockerfile은 이미지를 만드는 방법을 정의합니다.

전체 흐름

Dockerfile

실행 환경을 만드는 방법

Docker Image

환경과 애플리케이션을 패키징

Docker Container

패키지를 바탕으로 서비스 실행

4. Dockerfile에서 확인해야 할 다섯 가지

강의에서 제시한 학습 목표를 기준으로 정리하면 다음과 같습니다.

항목	의미	확인할 내용
베이스 이미지	실행 환경의 출발점	적절한 Python 버전인가?
시스템 패키지	운영체제에 설치하는 프로그램	사진 처리 등에 필요한 패키지가 있는가?
시크릿 제외	민감한 정보를 이미지에 넣지 않음	API 키가 코드나 이미지에 포함되지 않았는가?
포트	서비스가 요청을 받는 통로	서버 플랫폼이 요구하는 포트로 실행하는가?
실행 사용자	프로그램을 실행하는 계정	불필요하게 관리자 권한을 사용하지 않는가?

특히 API 키와 같은 시크릿은 Dockerfile에 직접 적지 않는 것이 중요합니다. 이미지는 다른 곳으로 복사하거나 저장소에 배포할 수 있기 때문에, 민감한 정보가 포함되면 유출 위험이 커집니다.

API 키는 컨테이너를 실행할 때 환경 변수 등 안전한 설정 경로로 전달하고, 운영 환경에서는 Secret Manager와 같은 비밀정보 관리 기능을 활용할 수 있습니다.

5. 실습 과정 이해하기

이번 강의에서 수행할 작업은 크게 다섯 단계입니다.

1. 로그 확인

   staging에서 이미지 처리가 실패한 원인을 로그로 조사합니다.

2. Dockerfile 작성

   베이스 이미지, 시스템 패키지, 실행 명령 등을 정의합니다.

3. 로컬 빌드 및 실행

   Dockerfile로 이미지를 만들고 컨테이너를 실행해 기능이 정상 동작하는지 확인합니다.

4. 환경별 자동 배포

   Dockerfile이 포함된 저장소를 staging과 production에 배포합니다.

5. 빌드 기록 확인

   실제 배포 과정에서 Dockerfile이 사용되었는지 빌드 기록을 확인합니다.

여기서 중요한 점은 CI 테스트 통과가 모든 운영 환경에서 정상 동작한다는 뜻은 아니라는 것입니다. 테스트가 확인한 범위와 실제 서비스 실행 환경이 다를 수 있기 때문입니다.

6. AI 개발자가 이 내용을 알아야 하는 이유

AI 에이전트는 일반적인 웹 서비스 외에도 이미지 처리 도구, 문서 변환 프로그램, 브라우저 자동화 도구 등 다양한 외부 구성 요소를 사용할 수 있습니다.

예를 들어 앞으로 개발하실 서비스에 적용하면 다음과 같습니다.

- MSM(My-Story-Maker): 사용자의 글을 분석하고 전자책을 만드는 과정에서 문서 변환 도구가 필요할 수 있습니다.
- Travel Creator: 여행 일정표나 이미지 자료를 처리할 때 별도 시스템 패키지가 필요할 수 있습니다.
- 뉴스레터 에이전트: 로컬에서 성공한 자동화 작업이 서버에서도 같은 조건으로 실행되도록 관리해야 합니다.
- AI 컨설팅 서비스: 고객에게 제공하는 서비스의 배포와 운영을 안정적으로 관리해야 합니다.

Dockerfile을 이해하면 개발자가 아니더라도 개발자와 협업할 때 “코드가 맞는가?”뿐 아니라 “실행 환경이 제대로 구성되었는가?”라는 중요한 질문을 할 수 있습니다.

7. 학습 목표와 창업 준비의 연결

사용자님의 1인 AI 컨설팅 사업 관점에서 이번 강의의 의미는 단순히 Docker 명령어를 외우는 것이 아닙니다.

AI 서비스를 만드는 능력과 그 서비스를 안정적으로 운영하는 능력은 서로 다릅니다.

고객에게 AI 에이전트를 구축해 제공하려면 다음 세 가지를 구분할 수 있어야 합니다.

1. 개발 문제: 코드와 로직이 올바른가?
2. 환경 문제: 필요한 라이브러리와 시스템 패키지가 설치되어 있는가?
3. 배포 문제: staging과 production에 동일한 실행 조건이 적용되었는가?

이 세 가지를 구분하는 능력은 향후 AI 서비스 진단이나 외부 개발자 관리 업무에도 활용할 수 있습니다.

8. 반드시 기억해야 할 핵심 정리

- Dockerfile: 실행 환경을 어떻게 구성할지 정의하는 파일입니다.
- 이미지: 실행에 필요한 환경과 애플리케이션을 패키징한 것입니다.

- 컨테이너: 이미지를 바탕으로 실행한 인스턴스입니다.
- buildpacks: 소스 코드를 분석해 실행 환경을 자동 구성합니다.
- 시스템 패키지: 코드의 의존성 선언만으로 해결되지 않을 수 있으므로 직접 확인해야 합니다.
- 시크릿: API 키 등을 이미지에 포함하지 않고 실행 환경에서 안전하게 전달해야 합니다.
- CI 통과: 테스트가 성공했다는 뜻이지, 모든 배포 환경에서 반드시 성공한다는 뜻은 아닙니다.

9. 시작 질문 예시 답안

강의 첫 질문에 답한다면 다음처럼 정리할 수 있습니다.

내 컴퓨터에서는 잘 작동하지만 서버에서는 실패하는 경우를 생각해 볼 수 있습니다. 같은 코드라도 Python 버전이나 설치된 라이브러리가 다를 수 있고, 운영체제에 필요한 시스템 패키지가 설치되지 않았을 수도 있습니다. 또한 환경 변수나 API 키 같은 설정값, 포트 설정, 실행 권한이 다를 가능성도 있습니다. 따라서 코드만 비교할 것이 아니라 로그를 확인하고 로컬과 서버의 실행 환경을 비교해야 합니다.

10. 한 문장 요약

Dockerfile은 AI 서비스의 코드뿐 아니라 실행에 필요한 환경까지 명시하여, 내 컴퓨터에서 작동하던 서비스를 서버에서도 일관되게 실행하도록 돕는 설계도입니다.

Chapter02

챕터 02. 시작 상태 맞추기

1. 학습 목적

이번 강의의 핵심은 로컬 환경에서는 정상 작동하지만 staging 서버에서는 실패하는 이유를 로그로 찾아내고, 실행 환경의 차이가 문제의 원인임을 확인하는 것입니다.

챕터 01에서 Dockerfile이 실행 환경을 정의하는 설계도라는 것을 배웠다면, 이번 챕터에서는 왜 Dockerfile이 필요한지 실제 사례로 확인합니다.

이번 실습의 문제는 사진 속 글자를 읽는 OCR 기능입니다.

- 로컬 환경: OCR 정상 작동
- staging 서버: OCR 실패

- 기존 대화 기능: 정상 작동
- 배포 결과: 성공

이 네 가지 사실을 연결해 원인을 찾아가는 것이 학습의 핵심입니다.

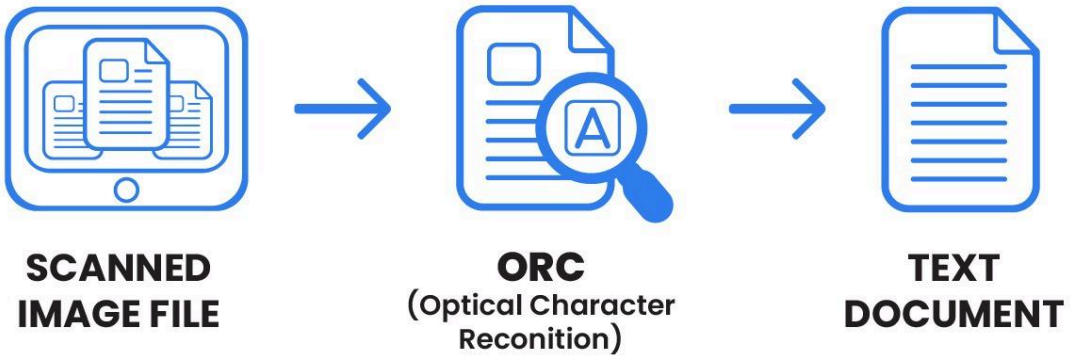
2. 중학생도 이해할 수 있는 설명

① Python 라이브러리만 설치하면 모든 기능이 작동할까요?

사진 속 글자를 읽는 기능을 책을 읽는 사람에 비유해 보겠습니다.

pytesseract — 통역을 요청하는 사람

Python 프로그램에서 OCR 기능을 사용하도록 연결해 주는 라이브러리입니다. 하지만 실제로 글자를 인식하는 작업은 다른 프로그램에 맡깁니다.



tesseract — 실제로 글자를 읽는 도구

이미지에서 글자를 인식하는 실제 프로그램입니다. 한국어를 읽으려면 한국어 언어 데이터도 필요합니다.

따라서 Python 라이브러리인 `pytesseract`만 설치했다고 해서 OCR 기능이 완성되는 것은 아닙니다.

다음 세 가지가 모두 필요합니다.

1. Python 라이브러리 `pytesseract`
2. 운영체제에 설치된 프로그램 `tesseract`

3. 한국어 인식을 위한 언어 데이터 `kor`

이처럼 Python 라이브러리 외부에 있는 프로그램이나 파일을 시스템 의존성이라고 이해하시면 됩니다.

3. 이번 실습의 핵심: 로컬과 staging의 차이

항목	로컬 환경	staging 환경
Python 코드	있음	있음
Python 라이브러리	있음	있음
tesseract 프로그램	직접 설치함	설치되지 않음
한국어 언어 데이터	설치했다면 있음	tesseract와 함께 없음
OCR 기능	정상 작동	실패

로컬 컴퓨터에 프로그램을 설치해도 그 설치 기록이 GitHub 저장소에 자동으로 들어가는 것은 아닙니다.

따라서 서버에서 같은 코드를 실행하더라도 필요한 프로그램이 없다면 실패할 수 있습니다.

핵심 원칙: 코드가 같다는 사실만으로 실행 환경까지 같다고 볼 수는 없습니다.

4. 실습 흐름 이해하기

이번 챕터는 기능을 켜고, 실패를 재현하고, 로그로 원인을 찾는 순서로 진행됩니다.

1

시작 상태 확인

배포 서비스와 브랜치, GitHub 환경 확인

2

로컬 OCR 실행

tesseract와 한국어 데이터 설치 후 테스트

3

staging에서 OCR 활성화

환경변수 OCR_ENABLED=true 추가

4

오류 로그 확인

tesseract 프로그램이 없다는 원인 확인

5

Dockerfile의 필요성 이해

실행 환경을 저장소에서 명시적으로 관리

환경변수를 추가할 때 주의할 점

staging 서비스에서 OCR을 켤 때는 다음 명령을 사용합니다.

```
gcloud run services update friend-agent-staging \
  --region us-central1 \
  --update-env-vars OCR_ENABLED=true
```

여기서 중요한 것은 `--update-env-vars`입니다.

- `-update-env-vars`: 지정한 환경변수를 추가하거나 해당 값만 갱신합니다.
- `-set-env-vars`: 지정한 환경변수 집합으로 설정을 교체하므로 기존 변수가 사라질 수 있습니다.

실습에서는 staging에만 기능을 활성화하며 production 서비스는 변경하지 않습니다.

또한 OCR 기능을 켜는 것과 OCR이 실제로 작동하도록 실행 환경을 갖추는 것은 서로 다른 작업입니다.

5. 로그를 통해 원인 찾기

staging에서 사진을 보냈을 때 화면에 오류 안내가 나타나더라도, 화면만 보고 정확한 원인을 단정하지 않습니다. Cloud Run 로그에서 오류가 발생한 시점의 기록을 확인해야 합니다.

이번 실습에서 확인할 핵심 기록은 다음과 같습니다.

- 오류 메시지: `tesseract` 프로그램을 찾을 수 없음
- 요청 기록: `POST /api/ocr`
- 상태 코드: `500`

이 기록을 종합하면 서비스 전체가 시작하지 못한 것이 아니라, 사진을 처리하는 요청이 들어왔을 때 필요한 프로그램을 실행하지 못한 것입니다.

이 차이가 중요합니다. 앱의 시작 단계에서는 OCR 프로그램을 검사하지 않았기 때문에 서비스는 정상적으로 시작될 수 있습니다. 실제 OCR 요청이 들어온 순간에야 문제가 드러난

것입니다.

6. 왜 CI 테스트는 문제를 발견하지 못했을까요?

CI는 코드를 변경할 때 자동으로 테스트를 실행하는 체계입니다. 하지만 테스트가 통과했다는 사실은 테스트에서 확인한 조건이 충족되었다는 의미이지, 모든 운영 환경이 완벽하다는 뜻은 아닙니다.

이번 사례에는 두 가지 이유가 있습니다.

1. OCR 테스트는 기능이 꺼져 있을 때 요청을 거절하는 동작을 확인했을 뿐, 실제 사진을 읽는 데 필요한 시스템 프로그램까지 검증하지 않았습니다.
2. GitHub의 테스트 러너와 Cloud Run의 staging 컨테이너는 서로 다른 실행 환경입니다.

따라서 CI가 성공했더라도 staging 이미지에 `tesseract`가 들어 있다는 보장은 없습니다.

이 문제를 해결하려면 실행 환경을 구성하는 방법을 코드 저장소에 명시해야 합니다. 그 역할을 하는 파일이 바로 Dockerfile입니다.

7. AI 개발자와 1인 창업 준비에 연결하기

이 강의는 단순히 OCR 오류를 해결하는 방법만 다루는 것이 아닙니다. AI 서비스를 개발하고 고객에게 제공할 때 필요한 운영 관점을 배우는 과정입니다.

사용자님이 구상 중인 프로젝트에도 연결됩니다.

- My-Story-Maker: 사용자의 글을 전자책으로 변환하거나 문서 파일을 처리할 때 외부 프로그램이 필요할 수 있습니다.
- Travel Creator: 이미지와 문서를 처리하는 기능을 서버에 배포할 때 실행 환경을 점검해야 합니다.
- AI 컨설팅 서비스: 고객의 서비스가 실패했을 때 코드 문제와 환경 문제를 구분해 진단할 수 있어야 합니다.

컨설턴트가 모든 오류를 직접 수정할 필요는 없습니다. 그러나 로그를 읽고 문제의 원인을 논리적으로 구분할 수 있다면 개발자와 훨씬 효과적으로 협업할 수 있습니다.

8. 객관식 퀴즈 정답

정답: 2번

앱은 시작할 때 `tesseract`의 존재를 확인하지 않았고, 사진 요청이 들어왔을 때 실제 프로그램을 호출하다가 실패했기 때문입니다.

다른 선택지가 틀린 이유도 확인해 보겠습니다.

- 1번: Cloud Run이 오류를 숨겨 준 것이 아닙니다. 요청 처리 과정에서 오류가 발생했습니다.
- 3번: staging에서 `OCR_ENABLED=true`로 기능을 켰기 때문에 OCR 요청이 실행되었습니다.
- 4번: 이번 오류는 한국어 데이터만 빠진 문제가 아니라, `tesseract` 프로그램 자체를 찾지 못한 문제입니다.

9. 반드시 기억해야 할 핵심 정리

- `pytesseract`는 OCR 프로그램을 호출하는 Python 라이브러리입니다.
- `tesseract`는 실제로 글자를 인식하는 외부 프로그램입니다.
- 한국어 OCR에는 `kor` 언어 데이터가 필요합니다.
- 로컬 컴퓨터에 설치한 프로그램은 서버에 자동으로 설치되지 않습니다.
- CI 테스트가 통과해도 실제 운영 환경의 의존성까지 검증되었다고 볼 수 없습니다.
- 로그는 오류가 발생한 위치와 원인을 좁히는 데 중요한 근거입니다.
- Dockerfile을 사용하면 필요한 시스템 패키지를 포함한 실행 환경을 명시적으로 정의할 수 있습니다.

10. 한 문장 요약

이번 챕터는 로컬과 서버의 실행 환경이 다르면 동일한 코드도 실패할 수 있음을 확인하고, 로그를 근거로 원인을 진단해 Dockerfile의 필요성을 이해하는 과정입니다.

Chapter03

챕터 03. 이미지와 컨테이너의 차이

1. 학습 목적

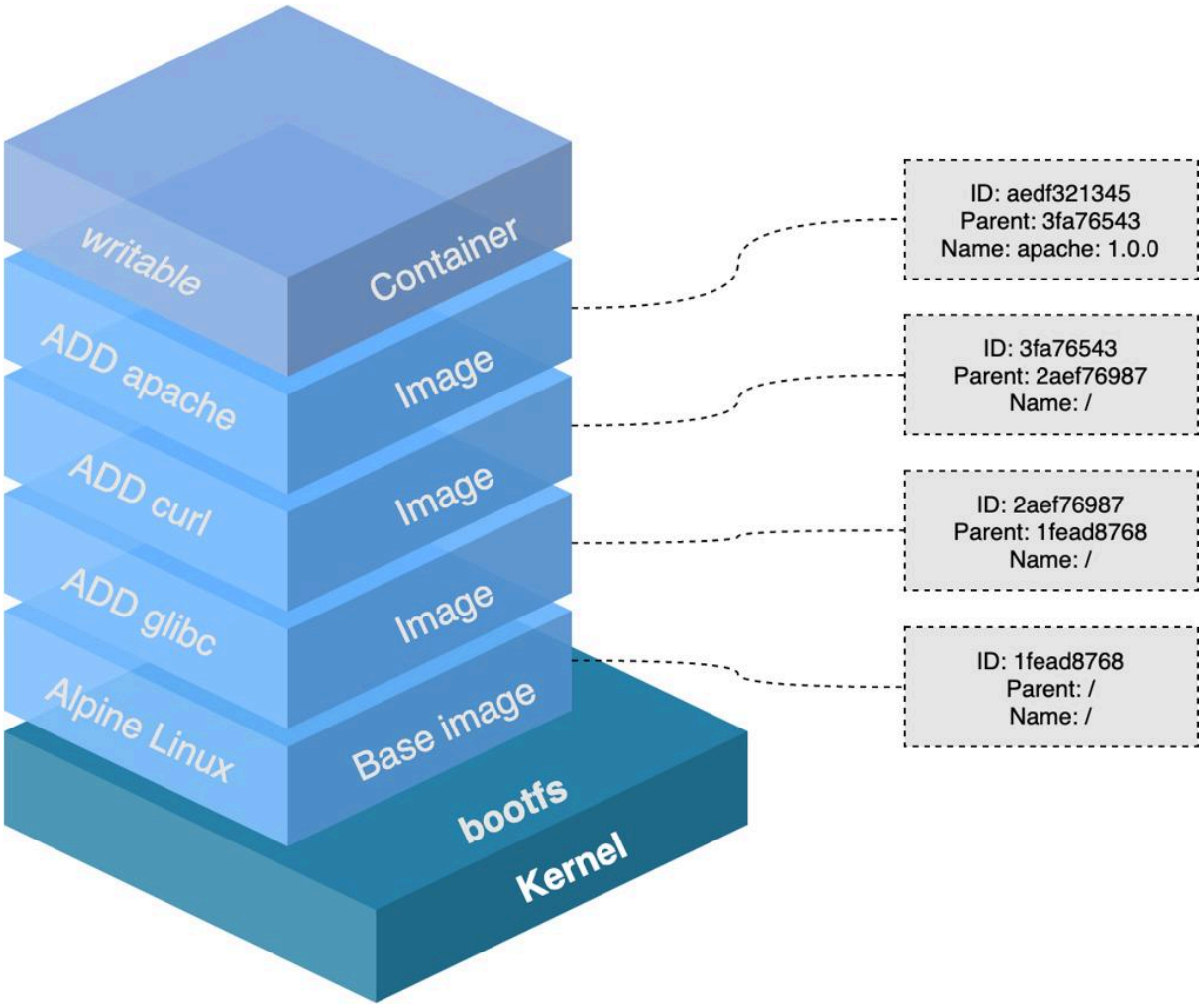
이번 강의의 핵심은 Docker 이미지와 컨테이너의 차이를 이해하고, 컨테이너의 파일·네트워크·실행 상태가 어떻게 분리되는지 직접 확인하는 것입니다.

챕터 01에서는 Dockerfile이 실행 환경을 정의하는 설계도라고 배웠고, 챕터 02에서는 로컬과 서버의 실행 환경 차이 때문에 OCR 기능이 실패하는 상황을 살펴봤습니다.

이번에는 Dockerfile을 작성하기 전에 Docker가 실제로 다루는 기본 개념을 익힙니다.

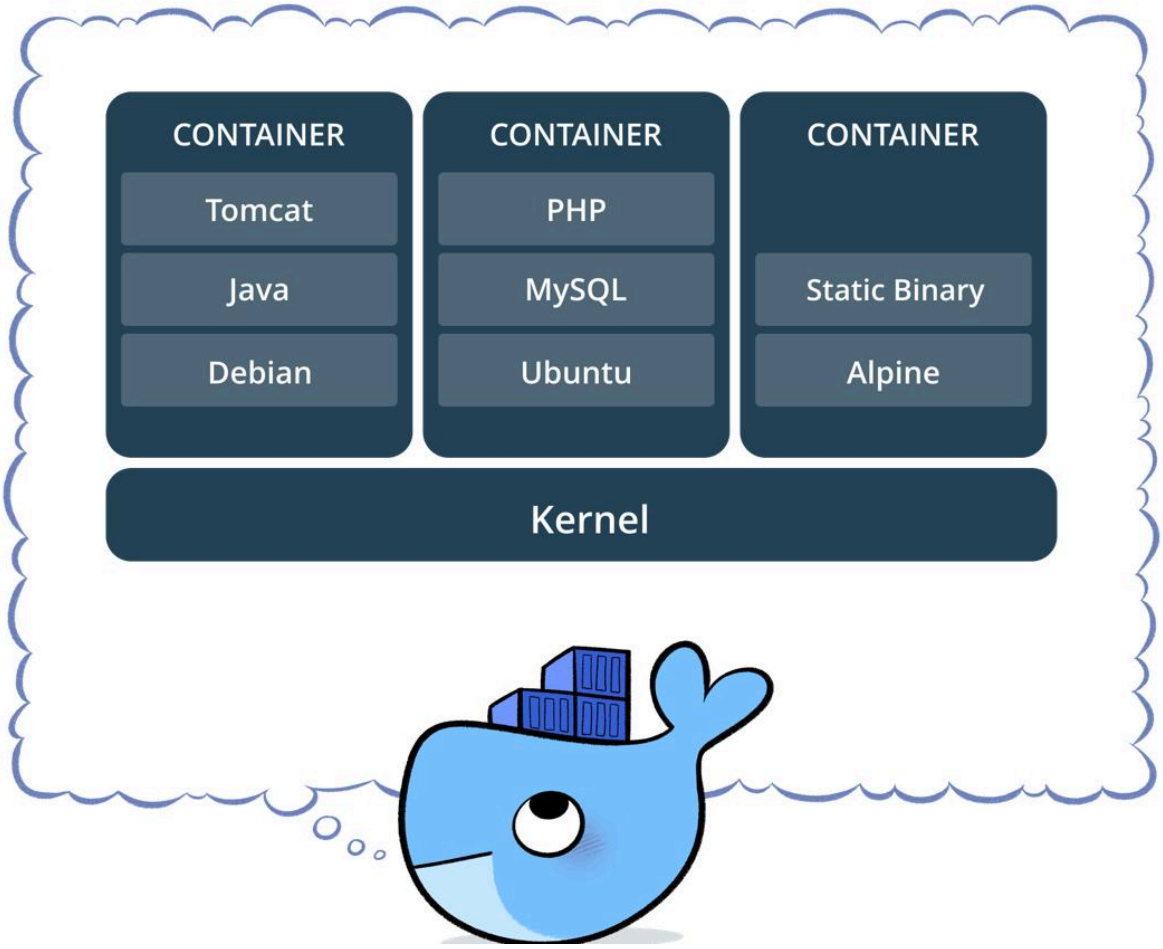
2. 중학생도 이해할 수 있는 설명

① 이미지와 컨테이너는 무엇이 다른가요?



이미지(Image) = 실행 준비물

운영체제 기본 파일, 프로그램, 라이브러리, 애플리케이션 코드와 시작 명령 등을 묶어 둔 패키지입니다. 같은 이미지를 여러 번 실행할 수 있습니다.



컨테이너(Container) = 실행된 프로그램 환경

이미지를 바탕으로 실제 실행한 인스턴스입니다. 컨테이너마다 실행 상태와 메모리, 기본적으로 분리된 파일 변경 사항을 가집니다.

비유하면 이미지가 붕어빵을 만들기 위한 준비물이라면, 컨테이너는 그 준비물을 이용해 실제로 만들어 낸 붕어빵입니다.

중요한 점은 컨테이너 안에서 파일을 변경해도 원본 이미지가 자동으로 바뀌지는 않는다는 것입니다.

3. Docker와 Cloud Run의 관계

이번 강의에서는 Docker의 개념을 Google Cloud Run의 운영 방식과 연결합니다.

Docker 개념	쉽게 이해하기	Cloud Run에서 대응하는 개념
이미지	실행 준비물 패키지	배포할 컨테이너 이미지
컨테이너	이미지를 실행한 인스턴스	인스턴스 안에서 실행되는 컨테이너

Docker 개념	쉽게 이해하기	Cloud Run에서 대응하는 개념
리비전(Revision)	특정 설정으로 배포한 서비스 버전	이미지와 환경변수, 타임아웃 등 설정을 묶은 버전
인스턴스 (Instance)	실제로 실행 중인 작업 공간	리비전을 바탕으로 실행되는 서비스 인스턴스

Docker 이미지 1개

코드 + 라이브러리 + 실행 환경

컨테이너 A

메모리·실행 상태 A

컨테이너 B

메모리·실행 상태 B

같은 이미지로 실행해도 컨테이너의 상태는 서로 분리됩니다.

이 개념이 왜 중요한가요?

예를 들어 에이전트 서비스가 대화 기록을 컨테이너의 메모리에만 저장한다고 가정하겠습니다.

1. 기존 컨테이너가 대화 기록을 메모리에 저장합니다.
2. 새로운 리비전을 배포합니다.
3. 새 리비전에서 새로운 컨테이너가 실행됩니다.
4. 새 컨테이너는 이전 컨테이너의 메모리를 자동으로 이어받지 않습니다.

따라서 이전 대화 기록이 사라질 수 있습니다.

서비스가 재시작되거나 인스턴스가 교체되어도 유지되어야 하는 데이터는 Supabase와 같은 외부 데이터베이스 또는 영속 저장소에 보관해야 합니다.

4. 실습 1 — Docker Desktop 준비하기

Docker Desktop은 Windows와 macOS에서 Docker를 사용하기 위한 프로그램입니다. Docker 엔진을 실행하고 이미지와 컨테이너를 관리하는 화면을 제공합니다.

터미널에서 다음 명령을 실행합니다.

```
docker --version
docker info
```

- `docker -version`: Docker 명령 도구의 버전을 확인합니다.
- `docker info`: Docker 엔진에 연결할 수 있는지 확인합니다.

첫 번째 명령은 성공하지만 두 번째 명령에서 엔진 연결 오류가 발생한다면 Docker Desktop이 실행 중인지 확인합니다.

Windows에서는 WSL 2와 가상화 설정이 필요할 수 있고, macOS에서는 컴퓨터의 칩 종류에 맞는 설치 파일을 선택해야 합니다.

5. 실습 2 — 공식 Python 이미지 실행하기

이번에는 Dockerfile을 직접 만들지 않고 공식 Python 이미지를 사용합니다.

```
docker run --rm python:3.13-slim-bookworm python --version

docker run --rm python:3.13-slim-bookworm sh -c "command -v tesseract || echo 'tessera
ct 없음'"
```

첫 번째 명령은 이미지 안의 Python 버전을 출력합니다. 두 번째 명령은 이미지 안에 `tesseract`가 있는지 확인합니다.

예상 결과는 다음과 같습니다.

- Python 버전이 출력됩니다.
- `tesseract 없음`이 출력됩니다.

여기서 `--rm`은 컨테이너가 실행을 마치면 자동으로 삭제하라는 옵션입니다. 이미지는 일반적으로 그대로 남아 다음 실행에서 재사용할 수 있습니다.

이 실습으로 확인할 수 있는 사실은 간단합니다. Python이 설치된 이미지라고 해서 OCR에 필요한 모든 외부 프로그램까지 설치되어 있는 것은 아닙니다.

6. 실습 3 — 컨테이너마다 파일이 분리되는지 확인하기

다음 명령을 차례대로 실행합니다.

```
# 첫 번째 컨테이너에서 파일 생성
docker run --name note-a python:3.13-slim-bookworm \
  sh -c "echo 메모 > /tmp/note.txt && cat /tmp/note.txt"

# 같은 이미지로 새 컨테이너 실행
docker run --rm python:3.13-slim-bookworm \
  sh -c "cat /tmp/note.txt || echo '파일 없음'"
```

```
# 첫 번째 컨테이너 삭제
docker rm note-a
```

예상 결과는 다음과 같습니다.

실행 단계	결과
첫 번째 컨테이너	`메모`
두 번째 컨테이너	`파일 없음`
첫 번째 컨테이너 삭제	`note-a` 컨테이너가 삭제됨

두 컨테이너는 같은 이미지로 시작했지만 첫 번째 컨테이너에서 만든 파일은 두 번째 컨테이너에 나타나지 않습니다.

단, 이 실습은 컨테이너 내부의 임시 파일을 사용하는 경우입니다. 볼륨이나 외부 저장소를 연결하면 여러 컨테이너가 같은 데이터를 공유하도록 구성할 수 있습니다.

Docker Desktop 화면에서는 무엇을 확인하나요?

- Images: 내려받은 `python:3.13-slim-bookworm` 이미지
- Containers: 이미지로 실행한 `note-a` 컨테이너

이미지와 컨테이너는 서로 다른 항목입니다. `docker rm note-a`는 컨테이너를 삭제하는 명령이며, 이미지를 삭제하는 명령이 아닙니다.

7. 실습 4 — 포트로 컨테이너에 요청 보내기

① 포트란 무엇인가요?

포트는 한 컴퓨터에서 여러 프로그램이 네트워크 요청을 구분하는 데 사용하는 번호입니다.

- `localhost`: 내 컴퓨터
- `localhost:9000`: 내 컴퓨터의 9000번 포트
- `localhost:9001`: 내 컴퓨터의 9001번 포트

컨테이너 안의 포트와 내 컴퓨터의 포트는 기본적으로 자동으로 연결되지 않습니다.
Docker의 `-p` 옵션으로 두 포트를 연결합니다.

② 포트 연결 명령

```
docker run -d --name web-a -p 9000:8000 \
  python:3.13-slim-bookworm python -m http.server 8000
```

```
docker run -d --name web-b -p 9001:8000 \
  python:3.13-slim-bookworm python -m http.server 8000

docker ps
```

- `p 9000:8000`에서 앞의 `9000`은 내 컴퓨터의 포트이고, 뒤의 `8000`은 컨테이너 내부 포트입니다.

브라우저: localhost:9000

web-a 컨테이너: 포트 8000

브라우저: localhost:9001

web-b 컨테이너: 포트 8000

브라우저에서 다음 두 주소를 각각 열면 모두 파일 목록이 표시됩니다.

- `http://localhost:9000`
- `http://localhost:9001`

두 컨테이너는 내부에서 같은 8000번 포트를 사용하지만, 내 컴퓨터에서는 각각 다른 포트로 연결되기 때문에 충돌하지 않습니다.

③ 같은 외부 포트를 사용할 수 있을까요?

다음 명령은 실패합니다.

```
docker run -d --name web-c -p 9000:8000 \
  python:3.13-slim-bookworm python -m http.server 8000
```

이미 `web-a`가 내 컴퓨터의 9000번 포트를 사용하고 있기 때문입니다.

실습이 끝나면 생성한 컨테이너를 삭제합니다.

```
docker rm -f web-a web-b web-c
```

이 명령은 해당 이름의 컨테이너를 강제로 종료하고 삭제합니다. 실행하기 전에 이름이 실습용 컨테이너인지 확인하시는 것이 좋습니다.

8. Cloud Run과 포트의 관계

Cloud Run은 컨테이너에 요청을 전달할 때 컨테이너가 수신해야 하는 포트를 환경변수 `PORT`로 알려 줍니다. 일반적으로 기본 포트는 8080입니다.

예를 들어 다음과 같이 실행했다면,

```
docker run -p 9000:8080 friend-agent:local
```

브라우저에서 접속할 주소는 다음과 같습니다.

`http://localhost:9000`

이때 내 컴퓨터의 9000번 포트가 컨테이너 내부의 8080번 포트로 연결됩니다.

따라서 에이전트 서비스는 포트 번호를 코드에 고정하기보다 `PORT` 환경변수를 읽어 해당 포트에서 요청을 기다리도록 구성하는 것이 중요합니다.

9. 시크릿을 이미지에 넣으면 안 되는 이유

Docker 이미지는 저장하고, 복사하고, 다른 서버에서 내려받아 실행할 수 있는 패키지입니다.

API 키와 같은 민감한 정보를 이미지 안에 넣으면 이미지를 받은 사람이 그 값을 확인할 위험이 있습니다.

따라서 다음 원칙을 기억해야 합니다.

- 이미지에는 애플리케이션과 실행에 필요한 파일을 넣습니다.
- API 키와 비밀번호는 이미지에 직접 포함하지 않습니다.
- 민감한 값은 실행 시 환경변수나 비밀정보 관리 서비스를 통해 전달합니다.
- 컨테이너 내부 메모리에만 저장한 데이터는 인스턴스 교체 후 사라질 수 있으므로 필요한 데이터는 외부 저장소에 보관합니다.

10. 객관식 퀴즈 정답

문제 1. 같은 이미지로 컨테이너 두 개를 실행하고 첫 번째 컨테이너에서 `/tmp/note.txt` 를 만들었다면?

정답: 2번

파일은 첫 번째 컨테이너의 파일 변경 사항이며, 원본 이미지나 두 번째 컨테이너에 자동으로 반영되지 않기 때문입니다.

문제 2. `docker run p 9000:8080 friend-agent:local` 로 실행했다면?

정답: 2번 — `http://localhost:9000`

앞의 9000은 내 컴퓨터의 포트이고 뒤의 8080은 컨테이너 내부의 포트이기 때문입니다.

11. AI 개발자와 1인 창업 준비에 연결하기

이번 강의는 앞으로 AI 서비스를 만들고 배포할 때 필요한 세 가지 운영 원칙을 알려 줍니다.

1. 실행 환경의 재현성: 같은 이미지를 사용해 일관된 실행 환경을 구성할 수 있습니다.
2. 데이터의 지속성: 컨테이너의 메모리나 임시 파일에만 데이터를 보관하면 재시작이나 교체 시 사라질 수 있습니다.
3. 서비스 접근성: 포트 연결을 이해해야 로컬에서 컨테이너 안의 웹 서비스에 접속할 수 있습니다.

이는 MSM, Travel Creator, 뉴스레터 에이전트와 같은 프로젝트를 실제 서비스로 배포할 때도 중요한 기초가 됩니다.

12. 반드시 기억해야 할 핵심 정리

- 이미지: 실행 환경과 애플리케이션을 묶어 둔 패키지입니다.
- 컨테이너: 이미지를 바탕으로 실행한 인스턴스입니다.
- 이미지 하나로 여러 컨테이너를 실행할 수 있습니다.
- 컨테이너의 파일 변경은 원본 이미지에 자동으로 반영되지 않습니다.
- 볼륨과 외부 데이터베이스를 사용하면 데이터의 지속성과 공유를 구성할 수 있습니다.
- `p 9000:8000`은 내 컴퓨터의 9000번 포트를 컨테이너의 8000번 포트에 연결합니다.
- Cloud Run은 `PORT` 환경변수를 통해 서비스가 수신할 포트를 알려 줍니다.
- API 키 등 시크릿은 이미지에 포함하지 않습니다.

13. 한 문장 요약

Docker 이미지는 실행 환경을 담은 패키지이고 컨테이너는 그 패키지를 실행한 독립적인 인스턴스이며, 둘의 차이를 이해해야 데이터·네트워크·배포를 올바르게 관리할 수 있습니다.

Chapter04

챕터 04. Dockerfile에 적을 것

1. 학습 목적

이번 강의의 핵심은 Dockerfile에 AI 서비스의 실행 환경을 명시하고, 안정성·보안·배포 일관성 관점에서 올바르게 작성되었는지 검토하는 것입니다.

앞선 강의에서 확인한 문제를 해결하기 위해 이번에는 실제 Dockerfile을 구성합니다.

- 챕터 01: Dockerfile은 실행 환경을 정의하는 설계도입니다.
- 챕터 02: staging에서 OCR이 실패한 원인은 `tesseract` 프로그램이 없었기 때문입니다.
- 챕터 03: 이미지와 컨테이너는 다르며, 컨테이너의 실행 상태와 파일 변경은 서로 분리됩니다.
- 챕터 04: 필요한 실행 환경을 Dockerfile에 직접 적고, 올바르게 작성했는지 검토합니다.

2. 중학생도 이해할 수 있는 설명

Dockerfile은 무엇인가요?

새로운 식당을 여러 곳에 열어도 같은 음식을 만들 수 있도록 조리법과 준비물 목록을 적어 놓는 것과 같습니다.

Dockerfile에는 다음 내용을 적습니다.

- 어떤 Python 환경에서 시작할 것인가?
- 어떤 프로그램과 라이브러리를 설치할 것인가?
- 소스 코드를 어디에 둘 것인가?
- 어떤 사용자 권한으로 실행할 것인가?
- 컨테이너가 시작되면 어떤 명령을 실행할 것인가?

이렇게 실행 환경을 명시하면 개발자의 컴퓨터에서만 잘 작동하는 문제를 줄일 수 있습니다.

3. Dockerfile의 주요 구성 요소

이번 강의에서 다루는 Dockerfile 예시를 살펴보겠습니다.

```
FROM python:3.13-slim-bookworm

RUN apt-get update \
    && apt-get install -y --no-install-recommends tesseract-ocr tesseract-ocr-kor \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd --create-home appuser
USER appuser

CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

각 부분이 무엇을 하는지 차례대로 살펴보겠습니다.

① 베이스 이미지 지정

```
FROM python:3.13-slim-bookworm
```

Docker 이미지의 출발점을 정합니다.

- `python:3.13`: Python 3.13 버전을 사용합니다.
- `slim`: 비교적 작은 이미지입니다.
- `bookworm`: Debian 운영체제 기반입니다.

`python:latest` 대신 구체적인 버전을 지정하면 빌드 시점에 따라 Python 버전이 달라질 위험을 줄일 수 있습니다.

② 시스템 패키지 설치

```
RUN apt-get update \
    && apt-get install -y --no-install-recommends tesseract-ocr tesseract-ocr-kor \
    && rm -rf /var/lib/apt/lists/*
```

이 부분은 앞선 강의의 OCR 오류를 해결하는 핵심입니다.

- `apt-get update`: 설치할 수 있는 패키지 목록을 갱신합니다.
- `apt-get install`: OCR 프로그램과 한국어 언어 데이터를 설치합니다.
- `rm rf /var/lib/apt/lists/*`: 설치 후 패키지 목록 캐시를 삭제해 이미지 크기를 줄입니다.

여기서 중요한 점은 `pytesseract`와 `tesseract-ocr`가 서로 다른 역할을 한다는 것입니다. 전자는 Python 라이브러리이고 후자는 운영체제 수준에서 설치하는 실제 OCR 프로그램입니다.

③ 작업 폴더 지정

```
WORKDIR /app
```

컨테이너 안에서 애플리케이션을 둘 기본 폴더를 `/app`으로 지정합니다.

이후 파일을 복사하거나 명령을 실행할 때 이 폴더를 기준으로 작업합니다.

④ 라이브러리를 먼저 설치

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

`requirements.txt`를 먼저 복사하고 Python 라이브러리를 설치합니다.

왜 코드보다 라이브러리를 먼저 설치할까요?

Docker는 이전 빌드에서 변경되지 않은 레이어를 재사용할 수 있습니다. 코드만 바뀌고 `requirements.txt`는 그대로라면 라이브러리를 다시 설치하지 않아도 될 가능성이 커집니다.

⑤ 나머지 코드 복사

```
COPY . .
```

현재 프로젝트의 파일을 이미지 내부의 `/app`으로 복사합니다.

단, `.dockerignore`에 적힌 파일과 폴더는 복사 대상에서 제외됩니다. 따라서 `.env`와 같은 민감한 파일을 제외하도록 설정하는 것이 중요합니다.

⑥ 일반 사용자로 실행

```
RUN useradd --create-home appuser
USER appuser
```

`appuser`라는 일반 사용자를 만들고 이후 명령을 해당 사용자 권한으로 실행하도록 지정합니다.

관리자 권한인 `root`로 애플리케이션을 실행하면 보안 문제가 발생했을 때 공격자가 할 수 있는 일이 더 많아질 수 있습니다. 일반 사용자로 실행하면 이러한 위험을 줄이는 데 도움이 됩니다.

⑦ 서비스 시작 명령

```
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

컨테이너가 시작될 때 FastAPI 애플리케이션을 실행합니다.

- `uvicorn app.main:app`: 애플리케이션 서버를 시작합니다.
- `-host 0.0.0.0`: 컨테이너 외부에서 들어오는 요청을 받을 수 있도록 바인딩합니다.
- `${PORT:-8080}`: `PORT` 환경변수가 있으면 해당 값을 사용하고, 없으면 8080을 사용합니다.

Cloud Run은 서비스가 수신할 포트를 `PORT` 환경변수로 전달하므로 이 값을 활용해야 합니다.

4. Dockerfile과 이미지 레이어

Dockerfile은 실행 환경을 구성하는 명령들을 순서대로 정의합니다. 각 명령으로 만들어지는 파일 변경 사항은 이미지 레이어로 관리될 수 있습니다.

레이어 4 — 애플리케이션 코드

자주 변경됨

레이어 3 — Python 라이브러리

requirements.txt가 바뀌면 재설치 가능

레이어 2 — 시스템 패키지

tesseract와 한국어 데이터

레이어 1 — 베이스 이미지

Python 3.13 + Debian

개념도이며, 실제 레이어 경계는 Docker 명령과 빌드 방식에 따라 달라질 수 있습니다.

이를 통해 알 수 있는 중요한 개념이 **레이어 캐시(layer cache)**입니다.

Docker는 이전 빌드와 비교해 변경되지 않은 단계의 결과를 재사용할 수 있습니다. 그래서 자주 바뀌는 코드는 아래쪽에 두고, 상대적으로 변경이 적은 시스템 패키지와 라이브러리 설치는 위쪽에 배치합니다.

이렇게 하면 반복 빌드에 걸리는 시간과 불필요한 작업을 줄일 수 있습니다.

5. `.dockerignore` 와 보안

Dockerfile이 있어도 이미지에 불필요하거나 민감한 파일이 들어가면 문제가 생깁니다.

`.dockerignore`는 Docker 빌드에 사용할 파일 중 복사하지 않을 대상을 지정합니다.

예를 들면 다음과 같습니다.

```
.env
.venv
.git
tests
docs
__pycache__
*.pyc
```

각 항목의 목적은 다음과 같습니다.

- `.env`: API 키 등 민감한 환경변수가 포함될 수 있습니다.
- `.venv`: 로컬 가상환경 파일은 이미지에 불필요합니다.
- `.git`: Git 이력과 내부 저장소 정보가 이미지에 포함되는 것을 방지합니다.
- `tests`, `docs`: 실행에 필요하지 않은 테스트 파일과 문서를 제외할 수 있습니다.

특히 `.dockerignore`에 `.env`를 넣고, API 키를 이미지에 직접 포함하지 않는 것이 중요합니다.

`.dockerignore`는 민감한 파일이 이미지에 들어가는 위험을 줄이는 장치이지, 시크릿 관리 자체를 대신하지는 않습니다.

6. Dockerfile 검토 체크리스트

확인 항목	올바른 기준	잘못 작성했을 때의 위험
베이스 이미지	`python:3.13-slim-bookworm`처럼 구체적으로 지정	버전 변경으로 실행 결과가 달라질 수 있음
시스템 패키지	`tesseract-ocr`, `tesseract-ocr-kor` 설치	OCR 기능 실패
민감한 파일	`.dockerignore`에 `.env` 포함	API 키 등 민감한 값이 이미지에 들어갈 수 있음
실행 포트	`PORT` 환경변수 사용	Cloud Run이 기대하는 포트에서 서비스가 대기하지 않을 수 있음
실행 사용자	`USER appuser` 등 일반 사용자 지정	불필요한 관리자 권한으로 보안 위험 증가

또한 기존의 `Procfile`과 `.python-version`은 buildpacks 방식에서 사용하던 설정입니다. Dockerfile로 빌드한다면 베이스 이미지와 `CMD`가 각각 Python 환경과 시작 명령을 결정합니다. 기존 파일을 반드시 삭제할 필요는 없지만, 설정이 서로 달라 혼란을 일으키지 않는지 확인해야 합니다.

7. AI 개발자와 1인 창업 준비에 연결하기

이번 강의는 단순히 Docker 문법을 외우는 수업이 아닙니다. AI 서비스를 반복해서 배포할 수 있도록 실행 조건을 명시하고, 보안과 운영 안정성을 함께 고려하는 방법을 배우는 과정입니다.

사용자님이 준비하시는 서비스에도 다음과 같이 적용할 수 있습니다.

- MSM: 글 분석, 문서 변환, 전자책 생성 등에 필요한 프로그램과 라이브러리를 실행 환경에 포함합니다.
- Travel Creator: 이미지·문서 처리 기능을 서버에 배포할 때 필요한 시스템 의존성을 관리합니다.
- 뉴스레터 에이전트: 로컬과 서버에서 같은 실행 환경을 유지해 자동화 작업의 차이를 줄입니다.
- AI 컨설팅: 개발자가 작성한 Dockerfile을 검토해 실행 환경 누락, 보안 위험, 배포 설정 문제를 찾아낼 수 있습니다.

코딩 에이전트에게 Dockerfile 작성을 맡길 수는 있지만, 어떤 Python 버전을 고정할지, 어떤 권한으로 실행할지, 어떤 파일을 제외할지에 대한 판단은 여전히 필요합니다.

8. 주관식 퀴즈 예시 답안

문제에서 제시한 Dockerfile은 세 가지 항목을 수정해야 합니다.

1. `FROM python:latest`

   `latest`는 구체적인 버전을 고정하지 않으므로, 빌드 시점에 따라 서로 다른 Python 환경이 사용될 수 있습니다. `FROM python:3.13-slim-bookworm`처럼 버전을 구체적으로 지정하는 것이 좋습니다.

2. **고정된 8000번 포트를 사용하는 CMD**

   Cloud Run이 전달하는 `PORT` 환경변수를 사용하지 않으므로, 서버가 기대하는 포트와 실제 애플리케이션이 대기하는 포트가 달라질 수 있습니다. `${PORT:-8080}`을 사용하도록 수정합니다.

3. `USER` **줄이 없음**

별도 사용자 지정이 없다면 애플리케이션이 기본적으로 root 권한으로 실행될 수 있습니다. `appuser` 같은 일반 사용자를 만들고 `USER appuser`로 실행하도록 수정하면 불필요한 권한을 줄일 수 있습니다.

결론: 베이스 이미지 버전 고정, `PORT` 환경변수 사용, non-root 실행은 각각 재현성, 배포 호환성, 보안 측면에서 중요합니다.

9. 반드시 기억해야 할 핵심 정리

- `FROM`: 실행 환경의 출발점을 지정합니다.
- `RUN`: 이미지 빌드 중 명령을 실행합니다.
- `WORKDIR`: 작업 폴더를 지정합니다.
- `COPY`: 파일을 이미지 안으로 복사합니다.
- `USER`: 애플리케이션을 실행할 사용자를 지정합니다.
- `CMD`: 컨테이너 시작 시 실행할 기본 명령을 지정합니다.
- `.dockerignore`: 빌드에 포함하지 않을 파일을 지정합니다.
- 레이어 캐시는 변경되지 않은 빌드 단계의 결과를 재사용해 빌드 시간을 줄입니다.
- API 키 등 시크릿은 이미지에 포함하지 않습니다.
- Dockerfile은 실행 환경을 명시하지만, 실제 배포 후에는 로그와 테스트를 통해 정상 작동 여부를 검증해야 합니다.

10. 한 문장 요약

Dockerfile은 베이스 이미지부터 시스템 패키지, Python 라이브러리, 코드, 실행 사용자와 시작 명령까지 명시하여 AI 서비스의 실행 환경을 재현 가능하고 안전하게 만드는 설계도입니다.

Chapter05

/강의정리 챕터05. 이미지 빌드와 실행

1. 이번 강의의 목적

이번 강의의 핵심은 Dockerfile로 만든 이미지를 내 컴퓨터에서 직접 실행하고, 그 안에 필요한 프로그램과 파일이 제대로 들어 있는지 확인하는 것입니다.

앞선 챕터에서는 Dockerfile에 실행 환경을 어떻게 정의하는지 배웠습니다. 이번에는 그 설계도를 실제 이미지로 만들고, 컨테이너를 실행해 봅니다.

이번 실습에서 확인할 것은 네 가지입니다.

- Docker 이미지 안에 OCR 프로그램인 `tesseract`와 한국어 데이터가 들어 있는가?
- API 키가 이미지에 포함되지 않고 실행할 때 안전하게 전달되는가?
- 컨테이너가 올바른 사용자와 포트로 실행되는가?
- 이미지 빌드 시 레이어 캐시가 작동하여 불필요한 작업을 줄이는가?

2. 중학생도 이해할 수 있는 설명

2-1. Dockerfile, 이미지, 컨테이너의 관계

Dockerfile — 설계도

어떤 프로그램과 파일을 넣고 어떻게 실행할지 작성합니다.

Docker 이미지 — 완성된 실행 패키지

Python, tesseract, 라이브러리, 애플리케이션 코드 등을 담습니다.

컨테이너 — 실제 실행 중인 프로그램

이미지를 실행하여 API 서비스를 작동시킵니다.

비유하면 Dockerfile은 요리법, 이미지는 미리 준비한 밀키트, 컨테이너는 밀키트로 실제 요리를 만드는 과정입니다.

이번 강의에서는 밀키트를 직접 만들고, 실제로 요리해 본 다음, 재료와 조리 도구가 제대로 들어 있는지 검사하는 것입니다.

2-2. 로컬 컴퓨터에 설치한 프로그램이 아니라 이미지 안의 프로그램을 사용합니다

기존에는 내 컴퓨터에 설치된 `tesseract`가 사진을 읽었습니다. 하지만 서버에는 해당 프로그램이나 한국어 데이터가 없어서 OCR 기능이 실패할 수 있었습니다.

이번에는 Docker 이미지 안에 다음 항목을 함께 넣습니다.

- Python 실행 환경
- 애플리케이션 코드
- 필요한 Python 라이브러리
- `tesseract-ocr`

- `tesseract-ocr-kor` — 한국어 OCR 데이터

따라서 내 컴퓨터의 별도 설치 상태에 의존하지 않고 이미지에 포함된 실행 환경을 사용할 수 있습니다.

단, 이미지에 필요한 요소를 포함했다고 해서 모든 환경에서 무조건 동일하게 동작하는 것은 아닙니다. CPU 아키텍처, 환경변수, 외부 서비스 연결 등도 확인해야 합니다.

3. 핵심 실습: 이미지 빌드와 컨테이너 실행

3-1. 이미지 만들기

```
docker build -t friend-agent:local .
```

각 부분의 의미는 다음과 같습니다.

명령어	의미
`docker build`	Docker 이미지를 빌드합니다.
`-t friend-agent:local`	이미지에 이름과 태그를 붙입니다.
`.`	현재 폴더를 빌드에 사용할 대상으로 지정합니다.

첫 빌드는 베이스 이미지와 시스템 패키지를 다운로드하고 설치하므로 몇 분 걸릴 수 있습니다.

3-2. 컨테이너 실행하기

```
docker run --rm -p 8080:8080 \
  --env-file .env \
  -e OCR_ENABLED=true \
  friend-agent:local
```

각 옵션을 이해하는 것이 중요합니다.

옵션	역할
`--rm`	컨테이너가 종료되면 컨테이너를 자동으로 삭제합니다.
`-p 8080:8080`	내 컴퓨터의 8080 포트를 컨테이너의 8080 포트에 연결합니다.
`--env-file .env`	`.env` 파일의 환경변수를 컨테이너 실행 시 전달합니다.
`-e OCR_ENABLED=true`	OCR 기능을 켜는 환경변수를 추가합니다.
`friend-agent:local`	실행할 이미지 이름입니다.

주의: 로컬 컴퓨터의 8080 포트를 다른 서비스가 사용 중이면 포트 충돌이 발생할 수 있습니다. 기존 서비스를 확인하고 필요하면 먼저 종료해야 합니다.

컨테이너가 실행되면 브라우저에서 `http://localhost:8080`을 열어 `schedule.png`를 다시 보내 봅니다. 한국어 일정이 정상적으로 읽히는지 확인합니다.

4. API 키는 왜 이미지에 넣지 않을까요?

이번 강의에서 보안과 관련해 반드시 이해해야 할 부분입니다.

권장 방식: 실행할 때 전달하기

```
docker run --env-file .env friend-agent:local
```

이미지에는 키를 넣지 않고, 컨테이너를 실행할 때 환경변수로 전달합니다.

피해야 할 방식: 이미지에 키 복사하기

```
COPY .env .
```

키가 이미지에 포함되면 이미지를 공유하거나 저장소에 올릴 때 비밀정보가 노출될 위험이 있습니다.

여기서 `.dockerignore`도 중요합니다.

```
.env
.venv
.git
tests
docs
**/__pycache__/
```

이 설정은 지정된 파일과 폴더가 빌드 컨텍스트에 포함되는 것을 막는 데 사용합니다. 특히 `**/__pycache__/`는 하위 폴더의 Python 캐시 폴더까지 제외하기 위한 패턴입니다.

다만 두 가지를 구분해야 합니다.

- `.dockerignore`는 파일이 이미지 빌드에 포함되는 것을 방지하는 장치입니다.
- `-env-file`은 실행 시 환경변수를 전달하는 방법입니다.

또한 `.env` 파일에는 값을 감싸는 불필요한 따옴표를 넣지 않는 것이 좋습니다. `--env-file`이 따옴표를 항상 제거해 주는 것은 아니기 때문입니다.

추가로 기억할 점: 이미지에서 키를 제외해도 실행 중인 컨테이너의 환경이나 로그 등에 키가 노출될 가능성까지 없어지는 것은 아닙니다. 운영 서버에서는 Secret Manager 같은 비밀정보 관리 수단을 활용하는 것이 좋습니다.

5. 이미지 내부 검사하기

이미지가 빌드되었다고 끝이 아닙니다. 실제로 필요한 파일과 설정이 들어 있는지 확인해야 합니다.

5-1. 파일 목록 확인

```
docker run --rm friend-agent:local ls -a /app
```

확인할 항목은 다음과 같습니다.

- 있어야 할 파일: `app`, `static`, `requirements.txt`
- 없어야 할 파일: `.env`, `tests`, `docs`
- 불필요한 파일: `__pycache__`

`.env`가 발견된다면 이미지를 다른 곳에 배포하지 말고, `.dockerignore`를 수정한 뒤 다시 빌드해야 합니다. 실제 키가 이미지에 포함되었다면 키 교체도 검토해야 합니다.

5-2. 실행 사용자 확인

```
docker run --rm friend-agent:local whoami
```

결과가 `appuser`인지 확인합니다.

애플리케이션을 관리자 권한인 `root`로 실행하면 문제가 발생했을 때 피해 범위가 커질 수 있습니다. 필요한 권한만 부여하는 것이 보안의 기본입니다.

5-3. 이미지 크기 확인

```
docker image ls friend-agent:local
```

실습에서 이미지 크기는 약 400MB대로 나올 수 있습니다. 이는 베이스 이미지, tesseract, Python 라이브러리 등이 차지하는 용량 때문입니다.

이미지가 크면 다운로드와 배포에 더 많은 시간이 걸릴 수 있습니다. 다만 무조건 크기만 줄이기보다 필요한 기능과 보안, 실행 안정성을 함께 고려해야 합니다.

6. 레이어 캐시: 왜 두 번째 빌드는 빨라질까요?

Docker는 이미지 빌드 과정을 여러 단계인 **레이어(layer)**로 관리합니다.

예를 들어 Dockerfile이 다음과 같은 순서라면,

1. Python 베이스 이미지 지정
2. tesseract 설치
3. Python 라이브러리 설치
4. 애플리케이션 코드 복사

코드만 바뀌고 앞 단계의 입력이 그대로라면 Docker는 앞서 만든 레이어를 재사용할 수 있습니다.

빌드 결과 비교

첫 번째 빌드

이미지와 패키지를 다운로드하고 설치합니다. 시간이 오래 걸릴 수 있습니다.

변경 없이 다시 빌드

기존 레이어가 재사용되어 빠르게 완료될 수 있습니다.

README.md만 수정한 뒤 빌드

의존성 설치 단계는 캐시되고, 변경 영향을 받는 이후 단계는 다시 실행될 수 있습니다.

이 때문에 Dockerfile에서는 보통 자주 바뀌지 않는 의존성 설치 단계를 위쪽에 배치하고, 자주 바뀌는 소스 코드는 아래쪽에 배치합니다.

이번 실습에서 README.md에 빈 줄을 추가한 뒤 다시 빌드하면 캐시가 어떻게 작동하는지 관찰할 수 있습니다. 실습이 끝나면 README.md 변경 사항은 되돌립니다.

7. 왜 로컬에서 만든 이미지를 서버에 그대로 올리지 않을까요?

이번 강의에서는 로컬 이미지를 서버에 직접 배포하지 않습니다. 서버 배포 과정에서 같은 Dockerfile을 사용해 이미지를 다시 빌드합니다.

주요 이유는 두 가지입니다.

- CPU 아키텍처 차이: 로컬 컴퓨터와 서버가 `amd64`, `arm64` 등 서로 다른 아키텍처를 사용할 수 있습니다.

- 재현성과 추적 가능성: 서버 배포 과정에서 사용할 코드, 설정, 빌드 과정을 관리하면 어떤 버전이 배포되었는지 확인하기 쉽습니다.

실무에서는 단순히 Dockerfile이 같다는 것만으로 완전히 같은 결과가 보장되지는 않습니다. 베이스 이미지와 패키지 버전을 고정하고, 빌드·테스트·배포 과정을 자동화하는 것이 중요합니다.

확인이 끝나면 컨테이너가 실행 중인 터미널에서 `Ctrl+C`를 눌러 종료합니다. `--rm` 옵션을 사용했으므로 종료된 컨테이너는 자동으로 삭제됩니다. 이미지까지 자동으로 삭제되는 것은 아닙니다.

8. 이번 강의가 AI 개발자에게 중요한 이유

AI 서비스를 실제 사용자에게 제공하려면 코드를 작성하는 것만으로는 부족합니다. 실행 환경과 보안, 배포 과정을 함께 관리해야 합니다.

배운 내용	AI 서비스에서의 활용
이미지 빌드	개발·테스트·운영 환경의 차이를 줄이는 기반
컨테이너 실행	서비스를 독립된 실행 환경에서 시험
환경변수 전달	API 키와 기능 설정을 코드와 분리
이미지 내부 검사	필요한 파일과 실행 권한 검증
레이어 캐시	반복적인 개발·빌드 시간 절약
서버에서 재빌드	배포 과정의 재현성과 추적 가능성 향상

예를 들어 My-Story-Maker(MSM)에서 사용자의 글을 분석하는 AI 기능을 구현하거나, Travel Creator에서 여행 일정을 생성하는 서비스를 만들 때도 API 키 관리, 실행 환경, 배포 재현성은 동일하게 중요합니다.

특히 1인 AI 창업에서는 서비스 기능을 만드는 능력뿐 아니라, 내 컴퓨터에서 만든 서비스를 다른 환경에서도 안정적으로 실행할 수 있게 만드는 능력이 필요합니다.

9. 객관식 퀴즈 정답과 해설

정답: 3번

> 실행할 때 키를 전달하면 키가 이미지 파일에 남지 않아, 이미지가 저장소에 올라가거나 다른 사람에게 전달돼도 키가 따라가지 않습니다.

- 1번: 틀립니다. 두 방식 모두 컨테이너에서 키를 읽을 수 있지만, 키가 저장되는 위치와 노출 위험이 다릅니다.

- 2번: 틀립니다. 이미지가 변경되지 않는다는 사실은 이미지에 포함된 비밀정보를 안전하게 만들어 주지 않습니다.
- 3번: 맞습니다. 이미지와 비밀정보를 분리할 수 있어 이미지 공유 시 키가 함께 유출될 위험을 줄입니다.
- 4번: 틀립니다. `-env-file`은 비밀정보 관리 시스템을 완전히 대체하지 않습니다. 운영 환경에서는 별도의 비밀정보 관리가 필요할 수 있습니다.

10. 반드시 기억할 것

1. Dockerfile은 설계도, 이미지는 실행 패키지, 컨테이너는 실제 실행 환경입니다.
2. 이미지 안에 tesseract와 한국어 데이터를 넣으면 로컬 컴퓨터의 설치 상태에 대한 의존성을 줄일 수 있습니다.
3. API 키는 Dockerfile에 복사하지 말고 실행 시 환경변수로 전달합니다.
4. `.dockerignore`로 `.env`, 테스트 파일, 캐시 등 불필요한 파일이 이미지에 포함되지 않도록 합니다.
5. 컨테이너는 가능한 한 `root`가 아닌 최소 권한 사용자로 실행합니다.
6. 레이어 캐시는 빌드 시간을 줄이므로 변경 빈도가 낮은 설치 단계를 위쪽에 배치합니다.
7. 로컬 이미지가 실행된다고 해서 운영 서버에서도 반드시 성공하는 것은 아닙니다. 서버 환경과 배포 과정을 검증해야 합니다.

한 문장 요약

이번 강의는 Dockerfile로 이미지를 직접 빌드하고 실행한 뒤, 필요한 프로그램·파일·권한·환경변수·캐시가 제대로 구성되었는지 검증하는 방법을 배우는 과정입니다.

Chapter06

/강의정리 챕터06. Dockerfile을 기존 자동 배포로 올리기

1. 이번 강의의 목적

이번 강의에서는 지난 챕터에서 로컬 환경에서 검증한 Docker 이미지를 기존 GitHub 자동 배포 파이프라인에 연결하여 staging과 production 서버에 적용하는 방법을 배웁니다.

앞선 챕터까지는 내 컴퓨터에서 Docker 이미지가 정상적으로 실행되는지 확인했습니다. 이제 그 실행 환경을 실제 서버에 적용하고, OCR 기능이 운영 환경에서도 정상적으로 작동하는지 검증합니다.

이번 강의의 핵심은 네 가지입니다.

1. 기존 자동 배포 시스템에서 Dockerfile을 사용하도록 전환합니다.
2. GitHub PR과 CI를 통해 변경 사항을 안전하게 배포합니다.
3. staging에서 먼저 검증한 다음 production에 적용합니다.
4. Dockerfile을 사용해도 staging과 production의 이미지가 완전히 동일하다고 단정할 수 없다는 점을 이해합니다.

2. 중학생도 이해할 수 있는 설명

2-1. 기존 자동 배포와 Dockerfile의 관계

기존에는 Google Cloud가 buildpacks를 이용해 소스 코드를 분석하고 실행 환경을 구성했습니다.

그런데 buildpacks는 일반적인 Python 실행 환경은 준비해도, 우리가 추가로 필요한 `tesseract`와 한국어 OCR 데이터를 자동으로 알아서 설치해 주지는 못할 수 있습니다.

이번에는 Dockerfile에 실행 환경을 직접 정의합니다.

기존 방식과 변경 방식

기존

소스 코드 → buildpacks → 이미지 → Cloud Run

Google이 자동으로 실행 환경을 구성합니다.

변경

소스 코드 + Dockerfile → Dockerfile 기반 빌드 → 이미지 → Cloud Run

우리가 정의한 Python 버전, 시스템 패키지, 라이브러리 등을 바탕으로 이미지를 만듭니다.

비유하자면 기존 방식은 식당에 필요한 재료를 이야기하면 주방에서 알아서 준비해 주는 방식이고, 변경 방식은 재료 목록과 조리법을 직접 작성해 전달하는 방식입니다.

중요한 점은 자동 배포 시스템 자체를 새로 만드는 것이 아니라, 기존 배포 파이프라인의 빌드 방식을 바꾸는 것입니다.

2-2. Cloud Run 소스 배포에서도 Dockerfile을 사용할 수 있습니다

이번 실습은 소스 코드를 Cloud Run에 배포하는 기존 흐름을 유지합니다.

저장소 루트에 `Dockerfile`이 있으면 Cloud Run의 소스 배포 과정에서 buildpacks 대신 Dockerfile을 이용해 이미지를 만들 수 있습니다.

따라서 모든 배포 설정을 새로 만들 필요 없이, 필요한 파일을 추가하고 기존 워크플로의 한 부분을 수정하면 됩니다.

3. 핵심 변경 사항: `-clear-base-image`

이번 강의에서 가장 주의해야 할 설정입니다.

기존 buildpacks 방식에서는 Google이 관리하는 베이스 이미지의 보안 업데이트를 자동으로 적용하는 기능이 활성화되어 있을 수 있습니다. 그러나 Dockerfile 방식에서는 베이스 이미지를 Dockerfile의 `FROM` 줄에서 직접 정하므로 기존 자동 업데이트 설정과 충돌할 수 있습니다.

이때 배포 설정에서 `--clear-base-image` 옵션을 추가하여 기존의 자동 베이스 이미지 설정을 해제합니다.

예를 들어 `.github/workflows/deploy.yml`의 `flags:` 줄에 다음 옵션을 추가합니다.

```
flags: ... --clear-base-image
```

위 코드는 옵션을 추가하는 위치를 보여 주는 예시입니다. 실제 수정에서는 기존 `flags:` 줄의 다른 옵션을 유지하고 끝에 `--clear-base-image`만 추가해야 합니다.

이후 베이스 이미지의 보안 패치를 적용하려면 Dockerfile의 `FROM` 줄과 관련 빌드 과정을 관리해야 합니다. `FROM`에 버전을 고정했다면 새로운 버전으로 업데이트하고 다시 빌드해야 합니다.

실무에서 기억할 점: Dockerfile을 도입하면 실행 환경을 더 명확하게 통제할 수 있지만, 보안 패치 관리 책임도 커집니다.

4. 배포 과정에서 파일은 두 번 걸러집니다

이번 강의에서는 `.gcloudignore`와 `.dockerignore`의 차이도 중요합니다.

1단계: `.gcloudignore`

소스 코드를 Google Cloud로 올릴 때 어떤 파일을 제외할지 결정합니다.

2단계: Dockerfile과 `.dockerignore`

Docker 이미지 빌드 과정에서 어떤 파일을 빌드 컨텍스트에서 제외할지 결정합니다.

두 파일은 서로 다른 단계에서 작동합니다. 따라서 둘 다 점검해야 합니다.

특히 `.env`에는 API 키와 같은 비밀정보가 들어 있을 수 있으므로 다음을 확인해야 합니다.

- `.gcloudignore`에서 `.env`를 제외하는가?
- `.dockerignore`에서도 `.env`를 제외하는가?
- Git에 커밋할 파일 목록에 `.env`가 들어 있지 않은가?

이 세 가지는 서로 다른 확인 사항입니다. `.dockerignore`가 있다고 해서 Git 커밋에서 자동으로 제외되는 것은 아닙니다.

5. GitHub PR과 CI를 이용한 staging 배포

이번 강의에서는 변경 사항을 곧바로 운영 서버에 올리지 않고, 별도 브랜치와 PR을 활용합니다.

5-1. 배포 전 준비

수정 대상은 다음 세 파일입니다.

파일	변경 목적
`Dockerfile`	실행 환경과 필요한 시스템 패키지 정의
`.dockerignore`	이미지에 포함하지 않을 파일 지정
`.github/workflows/deploy.yml`	기존 배포 설정에 `--clear-base-image` 추가

먼저 수정 사항을 확인하고, 위 세 파일만 커밋 대상인지 검사합니다.

```
git add Dockerfile .dockerignore .github/workflows/deploy.yml
git commit -m "Dockerfile로 실행 환경 정하기"
git push -u origin add-dockerfile
gh pr create --base staging --title "Dockerfile 추가" --body "tesseract와 한국어 데이터를 포함한 실행 환경"
```

실무에서는 명령을 실행하기 전에 `git status`와 `git diff --cached`로 실제 변경 파일과 내용을 확인하는 습관이 중요합니다.

특히 코딩 에이전트에게 맡길 때는 다음 조건을 명시합니다.

- 지정된 세 파일만 수정·커밋합니다.
- `.env`가 커밋 대상에 없는지 확인합니다.
- `deploy.yml`에서는 `flags:` 줄 끝에 옵션 하나만 추가합니다.
- 강제 푸시 등 위험한 명령을 사용하지 않습니다.
- PR의 CI가 통과했는지 확인하고 결과를 보고합니다.

5-2. CI 통과 후 staging에 반영

```
gh pr checks --watch
gh pr merge --merge
```

첫 번째 명령은 PR의 상태 체크를 기다리며, 두 번째 명령은 PR을 병합합니다.

다만 명령이 성공했다고 해서 애플리케이션의 기능까지 정상이라는 뜻은 아닙니다. CI가 통과한 후에도 실제 staging 환경에서 OCR 기능을 시험해야 합니다.

6. Cloud Build 로그에서 Dockerfile 사용 여부 확인

배포가 실행되면 GitHub Actions 결과만 확인하는 것으로 끝내지 않습니다. 실제로 Google Cloud에서 어떤 방식으로 이미지가 빌드되었는지 확인해야 합니다.

다음 순서로 점검합니다.

1. GitHub 저장소의 Actions 탭에서 배포 워크플로가 성공했는지 확인합니다.
2. Google Cloud 콘솔의 Cloud Build → 기록(History)에서 최근 빌드를 엽니다.
3. 로그에서 `FROM python:3.13-slim-bookworm`에 해당하는 단계가 있는지 확인합니다.
4. `tesseract-ocr`, `tesseract-ocr-kor` 설치 단계가 나타나는지 확인합니다.
5. Cloud Run의 `friend-agent-staging` 리비전에서 새 리비전이 정상적으로 배포되고 트래픽을 받는지 확인합니다.

왜 로그를 확인할까요?

Dockerfile을 커밋했더라도 실제 빌드에 Dockerfile이 사용되지 않았다면 원하는 실행 환경이 만들어지지 않습니다.

따라서 빌드 로그는 단순히 배포가 성공했는지를 넘어, 우리가 의도한 방식으로 이미지가 만들어졌는지 확인하는 증거입니다.

7. staging에서 OCR 검증하기

staging 배포가 끝나면 테스트 환경 주소를 열고 이전에 사용했던 동일한 `schedule.png`를 다시 전송합니다.

확인할 사항은 다음과 같습니다.

- 사진 속 한국어 일정 세 줄이 입력창에 들어오는가?
- 로그에 OCR 성공을 나타내는 기록이 남는가?
- 기존의 `tesseract 프로그램을 찾을 수 없습니다` 오류가 사라졌는가?

- 새 Cloud Run 리비전이 정상적으로 요청을 처리하는가?

이 단계에서 OCR이 성공하면, 이미지 안에 필요한 OCR 프로그램과 한국어 데이터가 포함되었다는 점을 실제 서버에서 검증할 수 있습니다.

단, OCR 결과가 정상이라는 사실만으로 모든 기능과 보안 설정이 검증된 것은 아닙니다. 다른 기능과 운영 조건은 별도로 테스트해야 합니다.

8. production 배포

staging 검증이 끝나면 `staging` 브랜치에서 `main` 브랜치로 PR을 생성합니다.

```
gh pr create --base main --head staging \
  --title "Dockerfile로 실행 환경 정하기" \
  --body "staging에서 OCR 확인"
```

이후 CI 통과 여부를 확인하고, 승인 절차에 따라 병합 및 production 배포를 진행합니다.

이번 실습에서는 production 배포가 끝난 후 OCR 기능을 활성화합니다.

```
gcloud run services update friend-agent \
  --region us-central1 \
  --update-env-vars OCR_ENABLED=true
```

이 명령은 `friend-agent` 서비스에 `OCR_ENABLED=true` 환경변수를 추가하거나 해당 값을 갱신합니다. 강의의 목적은 기존 환경변수를 모두 교체하는 것이 아니라 OCR 기능을 켜는 것입니다.

운영 환경에서 주의할 점: 실제 명령을 실행하기 전에는 서비스 이름과 리전이 정확한지 확인하고, 기존 환경변수에 영향이 없는지 점검해야 합니다. production 변경은 사용자의 승인과 배포 정책에 따라 진행하는 것이 안전합니다.

배포 후 production 주소에서도 같은 사진을 전송합니다. 로컬 컨테이너, staging, production에서 같은 결과가 나오는지 비교합니다.

9. Dockerfile이 같아도 이미지가 완전히 같지 않을 수 있습니다

이번 강의에서 특히 깊이 이해해야 할 개념입니다.

staging과 production은 각각 브랜치 병합 시점에 Cloud Build를 실행하여 이미지를 새로 만듭니다.

동일한 Dockerfile

staging

첫 번째 빌드

production

두 번째 빌드

Dockerfile이 같더라도 두 번의 빌드에서 내려받는 패키지 버전 등이 달라지면 이미지 내용이 달라질 수 있습니다.

예를 들어 Dockerfile에 다음과 같은 명령이 있다고 가정해 보겠습니다.

```
RUN apt-get update \
    && apt-get install -y tesseract-ocr
```

`apt-get update`는 저장소의 최신 패키지 정보를 가져옵니다. staging과 production을 서로 다른 시간에 빌드하면 설치되는 패키지 버전이 달라질 가능성이 있습니다.

따라서 다음 개념을 구분해야 합니다.

- Dockerfile의 일관성: 어떤 방식으로 이미지를 만들 것인지 명시합니다.
- 빌드 결과의 동일성: 실제 이미지에 들어간 모든 구성요소까지 동일한지를 의미합니다.

운영 수준의 배포 체계에서는 검증된 이미지를 그대로 승격하는 방식, 베이스 이미지와 패키지 버전 고정, 이미지 다이제스트 관리 등을 활용해 재현성과 추적 가능성을 높일 수 있습니다.

이번 강의는 Dockerfile로 실행 환경을 명시하는 첫 단계이며, 앞으로는 이미지 자체의 동일성을 보장하는 배포 설계로 발전시킬 수 있습니다.

10. 객관식 퀴즈 정답과 해설

정답: 2번

> Dockerfile이 저장소 루트에 `Dockerfile`이라는 이름으로 있는지, 그리고 `.gcloudignore`가 Dockerfile을 제외하고 있지 않은지 확인합니다.

이유는 다음과 같습니다.

- Dockerfile이 저장소 루트에 없거나 이름이 다르면 소스 배포 과정에서 기대한 Dockerfile을 찾지 못할 수 있습니다.
- `.gcloudignore`가 Dockerfile을 제외하면 Google Cloud에 소스가 올라갈 때 해당 파일이 전달되지 않을 수 있습니다.

- 따라서 빌드 로그에 tesseract 설치 단계가 없다면, 먼저 Dockerfile이 빌드 단계에 실제로 전달되었는지 확인하는 것이 합리적입니다.

나머지 선택지는 OCR 설치 단계가 빌드 로그에 없는 현상을 직접 설명하는 첫 번째 원인으로 보기 어렵습니다.

11. 사용자의 AI 창업 프로젝트와 연결하기

이번 강의는 향후 직접 만들 AI 서비스들을 운영 환경에 배포할 때도 활용할 수 있습니다.

프로젝트	적용할 수 있는 내용
My-Story-Maker (MSM)	글 분석·생성 서비스를 staging에서 검증한 뒤 운영 환경에 배포
Travel Creator	여행 일정 생성 기능의 실행 환경을 Dockerfile로 명시
뉴스레터 에이전트	자동 수집·분석·발행 기능의 배포 과정을 CI와 연결
AI 컨설팅 서비스	고객별 서비스의 실행 환경, 설정, 배포 이력을 관리

특히 1인 AI 창업에서는 개발자에게 모든 운영 업무를 맡기기 어렵습니다. GitHub PR, CI, Cloud Build 로그, Cloud Run 리비전의 관계를 이해하면 코딩 에이전트가 수행한 작업을 직접 검토하고 배포 위험을 줄이는 데 도움이 됩니다.

12. 반드시 기억할 것

1. Cloud Run 소스 배포에서 Dockerfile이 제공되면 해당 Dockerfile을 이용해 이미지를 빌드할 수 있습니다.
2. 기존 buildpacks 기반 배포의 베이스 이미지 자동 업데이트 설정은 Dockerfile 방식과 충돌할 수 있으므로 `-clear-base-image` 옵션을 검토합니다.
3. `.gcloudignore`는 소스 업로드 단계, `.dockerignore`는 Docker 빌드 컨텍스트 단계에서 파일을 제외합니다.
4. 배포 성공 여부뿐 아니라 Cloud Build 로그에서 실제 Dockerfile과 tesseract 설치 단계가 실행되었는지 확인합니다.
5. staging에서 먼저 OCR 기능을 검증하고, 승인 절차를 거쳐 production에 배포합니다.
6. 같은 Dockerfile을 사용하더라도 staging과 production에서 이미지를 별도로 빌드하면 결과물이 달라질 수 있습니다.
7. 운영 수준에서는 검증한 이미지를 그대로 승격하거나 이미지 다이제스트를 관리하는 방법을 고려할 수 있습니다.

한 문장 요약

이번 강의는 Dockerfile을 기존 GitHub 자동 배포 파이프라인에 연결하고, CI와 Cloud Build 로그를 통해 staging에서 검증한 뒤 production까지 안전하게 배포하는 방법을 배우는 과정입니다.

Chapter07

/강의정리 챕터07. 과제 구성 — 이미지와 컨테이너의 동작 이해하기

1. 이번 강의의 목적

이번 강의는 새로운 개념을 많이 배우기보다는, 지금까지 배운 Docker 지식을 직접 실험하고 다른 기능에도 적용해 보는 과정입니다.

핵심 질문은 세 가지입니다.

1. 컨테이너를 삭제하고 다시 실행하면 이전 대화와 기억은 어떻게 될까요?
2. 같은 Dockerfile로 이미지를 두 번 만들면 완전히 같은 이미지가 나올까요?
3. OCR처럼 운영체제에 별도 프로그램이 필요한 기능을 새로 만들 때, 어떤 순서로 개발하고 배포해야 할까요?

이번 과제는 단순히 명령어를 따라 하는 데 목적이 있지 않습니다. 문제가 발생한 이유를 스스로 설명하고, 로컬 환경에서 staging까지 같은 기능이 작동하도록 만드는 것이 핵심입니다.

2. 과제 1. 실행 중인 컨테이너 들여다보기

2-1. 컨테이너 안에서 직접 확인하기

지금까지는 컨테이너를 실행하고 서비스가 작동하는지만 확인했습니다. 이번에는 실행 중인 컨테이너 안에 들어 있는 파일과 실행 환경을 직접 살펴봅니다.

먼저 컨테이너를 백그라운드에서 실행합니다.

```
docker run -d --name friend-agent-test \
  -p 8080:8080 \
  --env-file .env \
  friend-agent:local
```

각 옵션의 의미는 다음과 같습니다.

옵션	의미
`-d`	터미널을 점유하지 않고 백그라운드에서 실행합니다.
`--name friend-agent-test`	컨테이너에 알아보기 쉬운 이름을 지정합니다.
`-p 8080:8080`	로컬 포트와 컨테이너 포트를 연결합니다.
`--env-file .env`	환경변수를 실행 시 전달합니다.

이제 컨테이너 안에서 명령을 실행합니다.

```
docker exec friend-agent-test whoami
docker exec friend-agent-test pwd
docker exec friend-agent-test ls
```

`docker exec`는 이미 실행 중인 컨테이너 안에서 추가 명령을 실행하는 기능입니다.

- `whoami`: 현재 실행 사용자 확인
- `pwd`: 현재 작업 폴더 확인
- `ls`: 현재 폴더의 파일 목록 확인

즉, 컨테이너를 새로 만들지 않고도 내부 환경을 점검할 수 있습니다.

2-2. 컨테이너를 삭제하면 대화는 어떻게 될까요?

브라우저에서 `http://localhost:8080`을 열고 에이전트와 대화를 나눈 다음, 컨테이너를 삭제하고 같은 이미지로 다시 실행합니다.

```
docker rm -f friend-agent-test

docker run -d --name friend-agent-test \
  -p 8080:8080 \
  --env-file .env \
  friend-agent:local
```

다시 접속하면 이전 대화가 사라져 있을 수 있습니다. 이번 실습의 에이전트는 대화와 기억을 컨테이너 안의 메모리에 보관하기 때문입니다.

friend-agent:local 이미지

프로그램과 실행 환경이 들어 있습니다.

컨테이너 A

대화 기록 보유

컨테이너 B

새로 시작, 기존 메모리 없음

이미지는 프로그램의 실행 환경을 담고 있지만, 실행 중 컨테이너의 메모리 상태까지 자동으로 보존하지는 않습니다.

여기서 중요한 구분이 있습니다.

- 이미지: 프로그램과 실행 환경을 담은 패키지
- 컨테이너 메모리: 실행 중에 만들어진 임시 상태
- 영속 저장소: 컨테이너가 종료되어도 데이터를 보존하는 데이터베이스나 파일 저장소

따라서 MSM처럼 사용자의 글과 인터뷰 기록을 보존해야 하는 서비스에서는 대화와 기억을 컨테이너 메모리에만 두면 안 됩니다. Supabase 같은 외부 데이터베이스에 저장하고, 새 컨테이너가 시작될 때 필요한 정보를 다시 읽도록 설계해야 합니다.

Cloud Run에서 새 리비전을 배포할 때도 마찬가지입니다. 새 컨테이너가 시작되더라도 기존 사용자 데이터를 유지하려면 별도의 저장 구조가 필요합니다.

반드시 기억할 것: 컨테이너는 실행 환경을 분리하지만, 데이터의 영속성을 자동으로 보장하지 않습니다.

3. 과제 2. 같은 Dockerfile로 두 번 빌드하기

3-1. 같은 설계도라면 결과도 같을까요?

이번에는 같은 Dockerfile로 이미지를 두 번 만듭니다. 단, 이전 빌드 결과를 재사용하지 않도록 `--no-cache`를 사용합니다.

```
docker build --no-cache -t friend-agent:try-a .
docker build --no-cache -t friend-agent:try-b .
```

- `-no-cache`는 Docker의 기존 빌드 레이어 캐시를 재사용하지 않도록 합니다. 다만 모든 외부 입력이 고정되는 것은 아닙니다. 빌드 과정에서 최신 패키지를 다운로드한다면 시점에 따라 결과가 달라질 수 있습니다.

이제 이미지 ID를 확인합니다.

```
docker image inspect --format "{{.Id}}" friend-agent:try-a
docker image inspect --format "{{.Id}}" friend-agent:try-b
```

두 이미지의 ID를 비교하면 됩니다.

3-2. 이미지 ID와 패키지 목록 비교

이미지 안에 설치된 Python 패키지 목록은 다음과 같이 확인할 수 있습니다.

```
docker run --rm --entrypoint pip friend-agent:try-a freeze
docker run --rm --entrypoint pip friend-agent:try-b freeze
```

두 결과를 비교할 때는 다음을 구분합니다.

비교 항목	의미
이미지 ID	이미지 전체 내용의 동일성을 확인하는 데 사용
`pip freeze` 결과	설치된 Python 패키지와 버전 비교
이미지 ID가 다르고 패키지 목록이 같음	Python 패키지는 같지만 이미지의 다른 내용은 다를 수 있음
패키지 목록도 다름	Python 의존성 구성에도 차이가 있음

이미지 ID는 이미지의 내용에 따라 달라집니다. 따라서 ID가 다르면 동일한 이미지가 아닙니다. 반대로 `pip freeze` 결과가 같다고 이미지 전체가 같다는 뜻은 아닙니다.

강의에서 제시한 비교 명령의 `diff <(...)` 형식은 macOS와 Linux의 Bash 계열 터미널에서 사용할 수 있습니다. Windows의 일반 PowerShell에서는 그대로 작동하지 않을 수 있으므로, Git Bash 등 Bash 환경을 이용하는 것이 편리합니다.

실험이 끝나면 다음 명령으로 실습용 이미지를 정리합니다.

```
docker rmi friend-agent:try-a friend-agent:try-b
```

3-3. 이 실험이 실제 배포에서 중요한 이유

앞선 챕터에서는 staging과 production이 같은 Dockerfile을 사용해도 각각 이미지를 새로 빌드한다고 배웠습니다.

이번 실험은 그 차이를 직접 확인하는 과정입니다.

예를 들어 다음과 같은 Dockerfile 명령은 실행 시점에 따라 설치 결과가 달라질 수 있습니다.

```
RUN apt-get update \
    && apt-get install -y tesseract-ocr
```

며칠 뒤 다시 빌드하면 패키지 저장소의 최신 버전이 달라져 이미지 내용이 바뀔 수 있습니다.

따라서 운영 서비스에서는 단순히 Dockerfile만 공유하는 것보다 다음 방법을 고려할 수 있습니다.

- 베이스 이미지와 의존성 버전을 고정합니다.
- 빌드 결과를 기록하고 이미지 다이제스트를 관리합니다.
- staging에서 검증한 이미지를 production으로 그대로 승격합니다.

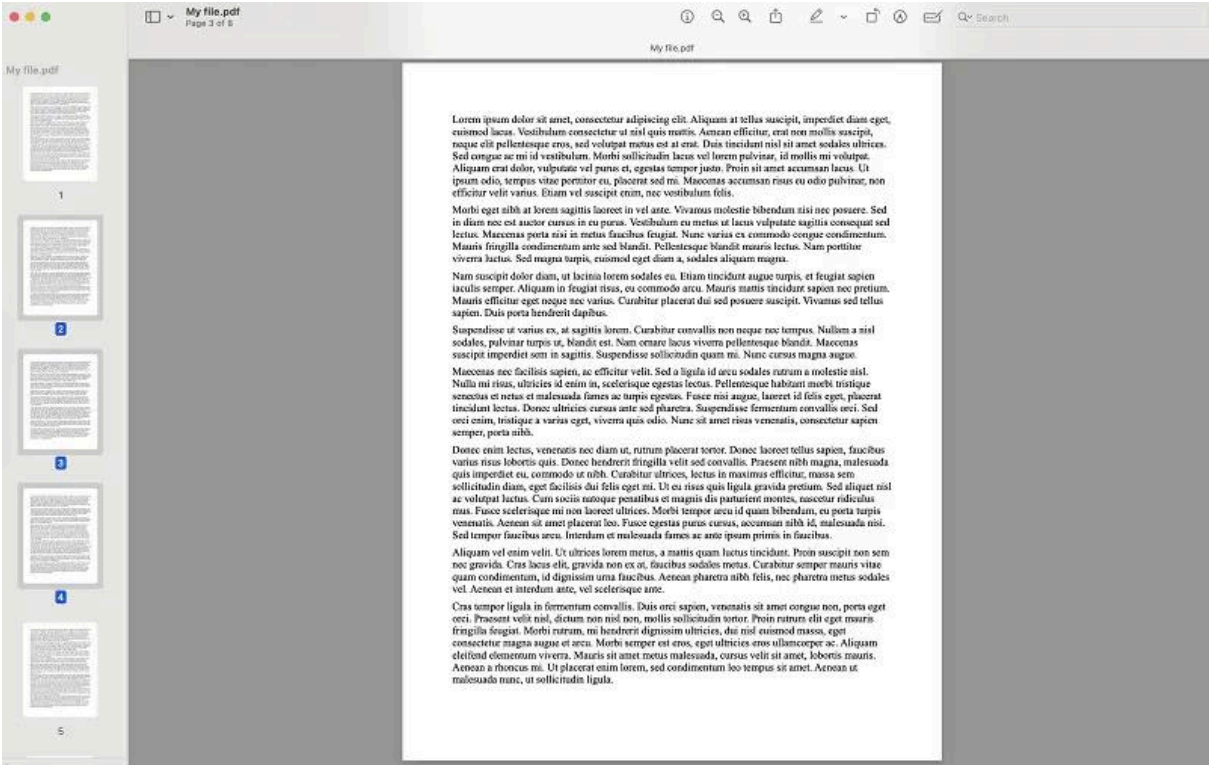
이렇게 하면 환경별로 별도 빌드할 때 발생할 수 있는 차이를 줄이고, 어떤 이미지가 운영 중인지 추적하기 쉬워집니다.

4. 과제 3. 시스템 프로그램이 필요한 기능 하나 더 붙이기

4-1. 오늘 배운 지식을 새로운 기능에 적용하기

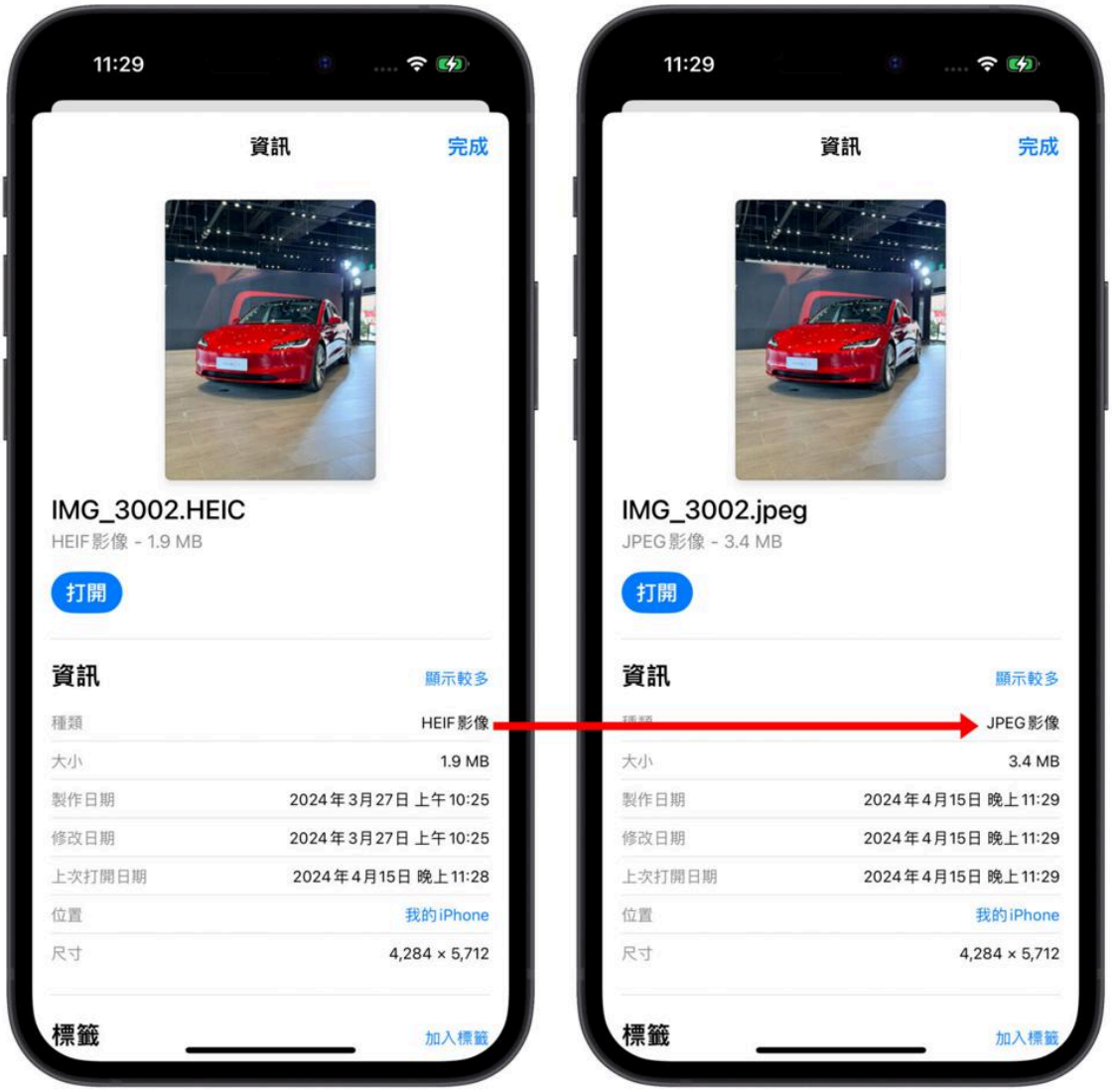
앞서 OCR 기능을 구현하면서 Python 라이브러리만 설치해서는 충분하지 않을 수 있다는 점을 배웠습니다. 실제 이미지에서 사진을 읽으려면 운영체제 수준의 `tesseract` 프로그램과 한국어 데이터도 필요했습니다.

이번 과제에서는 이 경험을 새로운 기능에 적용합니다.



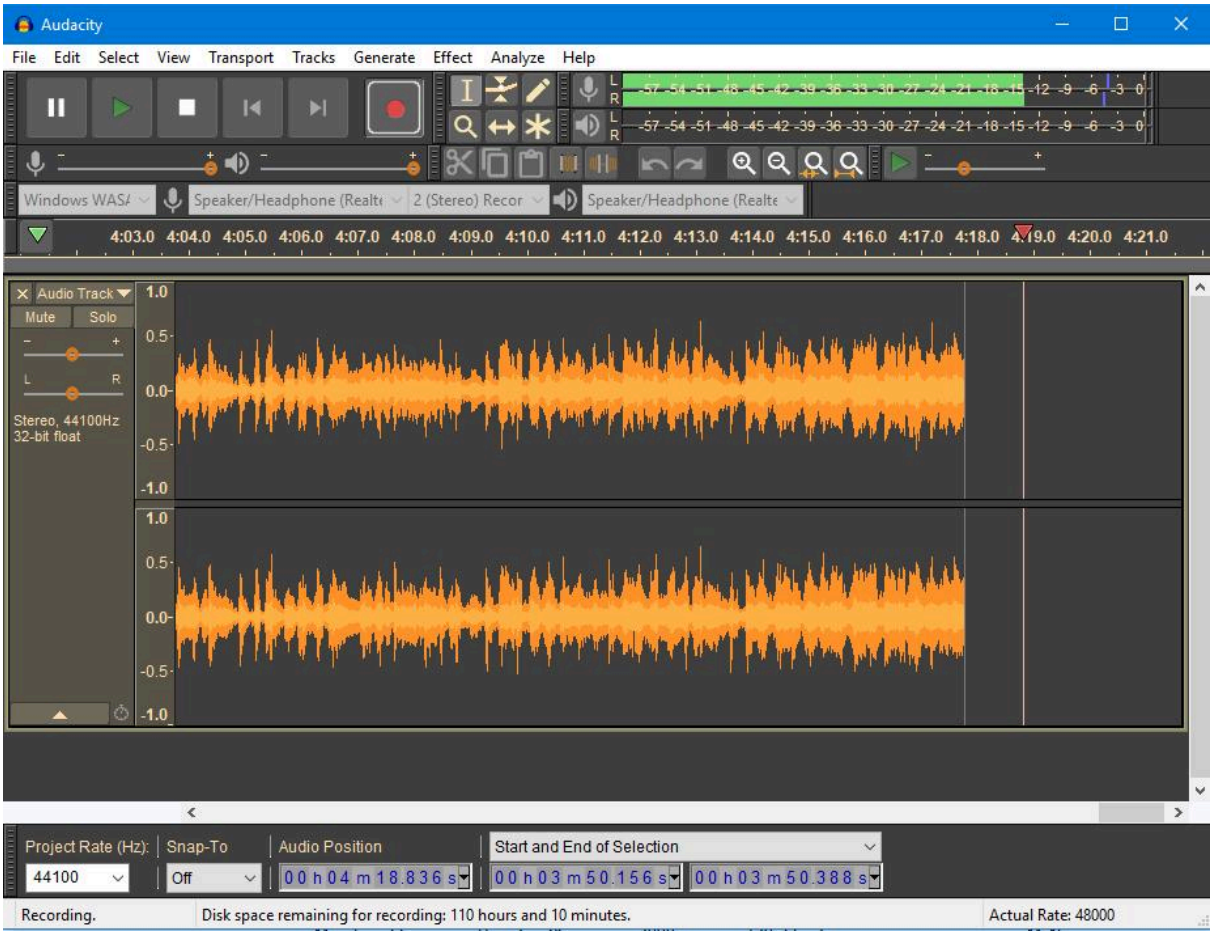
예시 1. PDF 공지문에서 글자 읽기

PDF 페이지를 이미지로 변환하는 기능이 필요하다면 Poppler 같은 도구를 검토할 수 있습니다.



예시 2. 아이폰 사진 형식 지원

HEIC 이미지를 처리할 때 libheif 계열 도구나 라이브러리가 필요한지 확인합니다.



예시 3. 음성 메모 처리

음성 파일 형식을 변환할 때 FFmpeg 같은 프로그램이 필요한지 확인합니다.



예시 4. 사진 속 QR 코드 읽기

QR 코드를 인식하는 기능에 zbar 계열 프로그램이 필요한지 검토합니다.

위 도구들은 가능한 예시이며, 실제로 필요한 시스템 패키지는 선택한 Python 라이브러리와 구현 방법에 따라 달라질 수 있습니다.

4-2. 과제 수행 순서

1. 기능 정하기: 어떤 기능을 만들고, 어느 화면에서 사용할지 결정합니다.
2. 필요한 시스템 프로그램 확인: 코딩 에이전트에게 Python 라이브러리 외에 별도로 설치해야 할 프로그램이 있는지 질문합니다.
3. 로컬에서 기능 구현: 작업 브랜치에서 기능을 구현하고, 내 컴퓨터에서 정상 작동하는지 확인합니다.
4. Dockerfile 수정: 필요한 시스템 프로그램을 설치하는 명령을 추가합니다. 이 단계는 먼저 필요한 내용을 스스로 판단한 다음 진행합니다.
5. 로컬 컨테이너 테스트: 로컬 환경과 같은 입력을 넣고 결과를 비교합니다.
6. staging 배포 및 테스트: PR을 통해 배포하고 staging에서도 동일한 입력으로 검증합니다.

이 과제에서 가장 중요한 것은 단순히 기능이 작동하는지 확인하는 데 그치지 않는 것입니다.

Dockerfile에 추가한 각 줄이 왜 필요한지 설명할 수 있어야 합니다.

예를 들어 FFmpeg가 필요한 음성 변환 기능을 선택했다면, 단순히 Python 라이브러리만 추가하는 것으로 충분한지 확인하고, 별도 실행 프로그램이 필요하다면 Dockerfile에서 이를 설치하도록 구성해야 합니다.

5. AI 코딩 에이전트에게 맡길 때의 원칙

이번 과제에는 사용자가 직접 판단해야 하는 부분과 에이전트에게 맡길 수 있는 부분이 함께 있습니다.

작업	사용자의 역할	코딩 에이전트의 역할
기능 기획	기능과 화면 결정	구현 방법과 의존성 조사 지원
시스템 의존성	필요한 이유 검토	별도 설치 프로그램 확인
기능 구현	결과 확인	코드 작성 및 로컬 테스트 지원
Dockerfile 변경	추가할 줄 결정	사용자가 결정한 내용을 반영
기존 기능 보호	변경 범위 승인	기존 기능과 워크플로 파일 보존
배포 검증	PR과 배포 승인	테스트 및 검증 결과 보고

이렇게 역할을 나누면 코딩 에이전트가 기존 기능이나 배포 워크플로까지 임의로 변경하는 위험을 줄일 수 있습니다.

특히 이번 과제에서는 기존 기능과 워크플로 파일은 바꾸지 말 것이라는 제약을 명시하는 것이 중요합니다.

6. 주관식 퀴즈 예시 답안

질문은 과제 3에서 선택한 기능과 Dockerfile에 추가한 줄, 그리고 해당 줄이 없었을 때 문제가 처음 발생했을 환경을 기록하는 것입니다.

예를 들어 음성 파일 변환 기능을 선택하고 FFmpeg가 필요하다고 확인했다면 다음과 같이 작성할 수 있습니다.

과제 3 수행 기록 예시

- **선택한 기능:** 음성 메모 파일을 텍스트 변환에 사용할 수 있도록 오디오 형식을 변환하는 기능
- **필요한 시스템 프로그램:** FFmpeg

- **Dockerfile에 추가한 줄:**

```
RUN apt-get update \
    && apt-get install -y --no-install-recommends ffmpeg \
    && rm -rf /var/lib/apt/lists/*
```

- **추가한 이유:** Python 코드가 FFmpeg 실행 프로그램을 호출하므로 컨테이너 내부에도 FFmpeg가 설치되어 있어야 합니다.
- **해당 줄이 없을 때 예상되는 문제:** 로컬 컴퓨터에 FFmpeg가 설치되어 있다면 로컬 환경에서는 작동할 수 있지만, Docker 이미지 내부에는 FFmpeg가 없어 로컬 컨테이너에서 처음 실패할 수 있습니다.
- **검증 방법:** 같은 음성 파일을 로컬 환경, 로컬 컨테이너, staging에 각각 입력해 변환 결과를 비교합니다.

참고: 위 내용은 작성 방법을 보여 주는 예시입니다. 실제 과제에서는 직접 선택하고 구현한 기능과 검증 결과를 기록해야 합니다.

7. 반드시 기억할 것

1. 컨테이너는 이미지에서 시작하지만, 실행 중에 만들어진 메모리 상태가 이미지에 자동으로 저장되지는 않습니다.
2. 컨테이너를 삭제하고 다시 실행하면 컨테이너 내부 메모리에만 저장한 대화와 기억은 사라집니다.
3. 데이터가 유지되어야 하는 AI 서비스는 외부 데이터베이스 등 영속 저장소를 사용해야 합니다.
4. 같은 Dockerfile로 빌드해도 별도로 생성된 이미지의 ID가 다를 수 있습니다.
5. `pip freeze`가 같더라도 운영체제 패키지나 파일 등 이미지 전체 내용까지 같다는 의미는 아닙니다.
6. 새로운 기능을 구현할 때는 Python 라이브러리뿐 아니라 시스템 프로그램 의존성도 확인해야 합니다.
7. 로컬 환경, 로컬 컨테이너, staging에서 동일한 입력과 결과를 비교하는 것이 중요한 검증 방법입니다.

8. 이번 과제와 1인 AI 창업의 연결

이번 챕터는 앞선 Docker 강의의 내용을 실제 서비스 개발 능력으로 전환하는 단계입니다.

- MSM: 사용자의 글과 인터뷰 내용을 컨테이너 메모리에만 저장하지 않고, 데이터베이스에 보존하는 구조를 설계할 수 있습니다.
- Travel Creator: 이미지 처리나 문서 읽기 기능을 추가할 때 시스템 의존성을 점검할 수 있습니다.
- 뉴스레터 에이전트: 동일한 실행 환경을 유지하고 배포 결과를 추적하는 데 활용할 수 있습니다.
- AI 컨설팅 사업: 코딩 에이전트가 작성한 코드와 Dockerfile을 검토하고, 고객에게 배포하기 전 필요한 테스트를 정의할 수 있습니다.

사용자께서 준비하시는 1인 AI 사업에서는 모든 코드를 직접 작성하는 것 못지않게, 에이전트에게 작업을 정확히 맡기고 결과를 검증하는 능력이 중요합니다.

한 문장 요약

이번 강의는 컨테이너의 임시 상태와 이미지의 차이를 실험하고, 동일한 Dockerfile의 빌드 결과를 비교하며, 시스템 프로그램이 필요한 새 기능을 로컬에서 staging까지 스스로 적용하는 훈련입니다.

Chapter08

챕터 08 강의 정리 — Docker 실행 환경과 이미지 관리

1. 학습 목적

이번 강의의 핵심은 코드가 정상이어도 실행 환경이 다르면 서비스가 실패할 수 있다는 점을 이해하고, Dockerfile을 통해 그 차이를 줄이는 것입니다.

지금까지는 로컬에서 작동하는 서비스를 서버에서도 작동하도록 만들었습니다. 이제 한 단계 더 나아가, 어떤 이미지가 실제로 배포되었는지 확인하고 테스트한 이미지와 운영 중인 이미지가 같은 것인지 관리하는 문제를 배우게 됩니다.

2. 중학생도 이해할 수 있는 설명

① 코드보다 실행 환경이 문제일 수 있습니다

요리법이 같아도 조리 도구나 재료가 다르면 결과가 달라질 수 있습니다.

- 코드: 요리법

- 실행 환경: 조리 도구와 재료
- Dockerfile: 필요한 도구와 재료를 적어 놓은 준비 목록

이번에는 로컬 컴퓨터에 설치된 `tesseract`가 서버에는 없어서 OCR 기능이 실패했습니다. Python 코드만으로는 시스템 프로그램까지 자동으로 준비되지 않았던 것입니다.

Dockerfile에 필요한 프로그램과 설치 방법을 기록하면, 이미지 빌드 과정에서 실행 환경을 함께 준비할 수 있습니다.

② 이미지와 컨테이너는 다릅니다

Docker 이미지

실행에 필요한 프로그램과 파일을 담은 패키지

이미지를 실행하면

컨테이너 A

실행 중인 서비스와 자체 상태

컨테이너 B

같은 이미지로 실행한 별도 인스턴스

두 컨테이너는 같은 이미지를 사용해도 메모리와 내부 파일 상태를 자동으로 공유하지 않습니다.

따라서 대화 기록이나 중요한 파일을 컨테이너 내부에만 저장하면 컨테이너가 교체될 때 데이터가 사라질 수 있습니다. 계속 보존해야 하는 데이터는 Supabase 같은 외부 저장소에 두어야 합니다.

③ Dockerfile이 있어도 이미지가 완전히 같다는 보장은 없습니다

Dockerfile은 이미지를 만드는 설계도입니다. 같은 설계도를 사용해도 빌드 시점이나 설치되는 패키지 버전이 다르면 결과물이 달라질 수 있습니다.

예를 들어 staging에서 테스트한 이미지와 production에 배포한 이미지를 각각 따로 만들었다면, 두 이미지가 동일하다고 단정할 수 없습니다.

이 문제를 해결하려면 이미지에 이름과 버전을 붙이고, 가능하면 한 번 검증한 이미지를 그대로 staging과 production에 배포하는 방식을 사용해야 합니다.

3. 오늘 배운 내용을 실제 서비스에 적용하면

확인 항목	왜 필요한가요?
`Dockerfile`	필요한 시스템 프로그램과 실행 환경을 기록합니다.

확인 항목	왜 필요한가요?
`.dockerignore`	`.env`, `.git`, 불필요한 파일이 이미지에 들어가는 것을 막습니다.
`PORT` 환경변수	Cloud Run이 지정한 포트로 서비스가 실행되도록 합니다.
비루트 사용자	컨테이너가 불필요하게 높은 권한으로 실행되는 것을 방지합니다.
빌드 기록	실제 이미지가 어떤 Dockerfile과 설치 단계로 만들어졌는지 확인합니다.
이미지 태그와 digest	이미지에 식별 정보를 부여하고 배포 대상을 추적합니다.
외부 데이터베이스	컨테이너가 교체되어도 대화 기록 등의 데이터를 유지합니다.

여기서 이미지의 태그(tag) 는 `friend-agent:v1.0`처럼 사람이 알아보기 쉬운 이름입니다. 반면 digest 는 이미지 내용을 식별하는 고유한 해시값입니다. 같은 태그가 나중에 다른 이미지로 변경될 수도 있지만, digest를 기준으로 하면 특정 이미지 내용을 더 정확하게 지정할 수 있습니다.

4. 주관식 퀴즈 모범 답안

다음은 제출란에 활용할 수 있는 답안입니다.

로컬에서는 직접 설치한 tesseract와 한국어 언어 데이터가 있어서 OCR이 작동했지만, staging에서는 실행 환경에 해당 프로그램이 없어 실패했습니다. 코드만으로 이미지를 만드는 buildpacks가 로컬에 설치된 시스템 프로그램까지 자동으로 포함하지 못한 것이 원인이었습니다.

이제 같은 문제가 발생하면 먼저 staging 로그에서 오류 메시지를 확인하고, Dockerfile에 필요한 시스템 프로그램과 언어 데이터가 설치되어 있는지 살펴보겠습니다. 다음으로 Dockerfile을 사용해 빌드했는지, 빌드 기록에 해당 설치 단계가 있는지 확인하겠습니다. 마지막으로 로컬 컨테이너에서 재현한 뒤 staging에서도 같은 입력으로 테스트하겠습니다.

5. AI 개발자가 이 내용을 알아야 하는 이유

AI 서비스는 모델이나 Python 코드만으로 작동하지 않습니다. OCR, PDF 변환, 음성 처리처럼 별도의 시스템 프로그램이 필요한 기능이 많습니다.

따라서 AI 서비스를 기획하고 운영할 때는 다음 세 가지를 구분할 수 있어야 합니다.

- 코드 문제: 프로그램 로직이나 입력 처리의 오류
- 환경 문제: 필요한 프로그램, 라이브러리, 설정의 누락
- 배포 문제: 테스트한 이미지와 실제 운영 이미지의 차이

이 구분을 할 수 있으면 오류가 발생했을 때 무작정 코드를 수정하는 대신 원인을 좁혀서 해결할 수 있습니다. 비개발자 AI 컨설턴트에게도 중요한 역량입니다. 개발팀과 대화할 때 문제의 원인과 검증 방법을 구체적으로 제시할 수 있기 때문입니다.

6. 다음 단계에서 해결할 과제

오늘 강의 마지막 부분에서 제시한 과제는 크게 세 가지입니다.

1. 이미지 추적: production에서 실행 중인 이미지가 어느 커밋으로 만들어졌는지 확인할 수 있도록 태그와 digest를 관리합니다.
2. 이미지 일관성: staging에서 검증한 이미지와 production에 배포한 이미지가 동일하도록 빌드·배포 흐름을 개선합니다.
3. 저장 공간 관리: 400MB대 이미지가 빌드할 때마다 쌓이는 문제를 확인하고, 불필요한 이미지와 레이어를 효율적으로 관리합니다.

특히 이미지 크기를 줄이는 것만큼 중요한 것은 검증한 결과물을 그대로 운영 환경에 배포하는 것입니다. 크기 최적화와 배포 추적은 별개의 과제로 나누어 접근하는 것이 좋습니다.

7. 반드시 기억할 것

- 로컬과 서버의 차이는 코드뿐 아니라 실행 환경에서도 발생합니다.
- Dockerfile은 실행 환경을 명시하지만, 별도로 빌드한 이미지가 완전히 같다는 보장은 없습니다.
- 컨테이너 내부 상태는 자동으로 공유되거나 영구 보존되지 않습니다.
- 이미지 태그는 이름표이고, digest는 특정 이미지 내용을 식별하는 기준입니다.
- staging에서 검증한 이미지와 production 이미지를 일치시키는 것이 배포 신뢰성을 높입니다.

8. 한 문장 요약

Dockerfile로 실행 환경을 표준화하는 단계에서 한 걸음 더 나아가, 실제로 검증한 이미지가 어떤 것인지 추적하고 동일한 이미지를 운영 환경에 배포하는 체계를 만들어야 합니다.